

KI-Dokumentation

Generiert am: 19.12.2025 um 17:28

Anzahl der Chats: 1

Legende & Hinweise

Definition: Was ist ein "Turn"?

Ein "Turn" bezeichnet einen einzelnen Interaktionsschritt im Dialog.



USER



MODEL

1. Python-Skript: Übersetzen, Suchen, Crawl, Indizieren

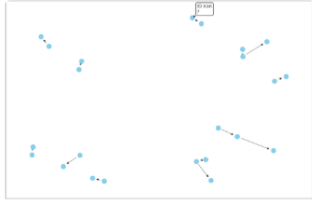
Gemini

Über Gemini Gemini App Abos Für Unternehmen [Anmelden](#)

Python-Skript: Übersetzen, Suchen, Crawl, Indizieren

<https://gemini.google.com/share/ac4a443a4c12>  

Erstellt mit Thinking mit 3 Pro 8. Dezember 2025 um 02:46 • Veröffentlicht: 16. Dezember 2025 um 01:31



Anstatt 20 weiterzusuchen und in der duckduckgo seite zu blättern, hat das Smart Crawl nach 7 validen ergebnissen aufgehört zu suchen.

Bei seiten mit viel javascript und asynchronem inhalt ist der Crawl gekommen.

[Anmelden](#)

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen.

Modell: Thinking mit | **Zeitpunkt:** 8. Dezember 2025 um 02:46
Original Chat Link (<https://gemini.google.com/share/ac4a443a4c12>)

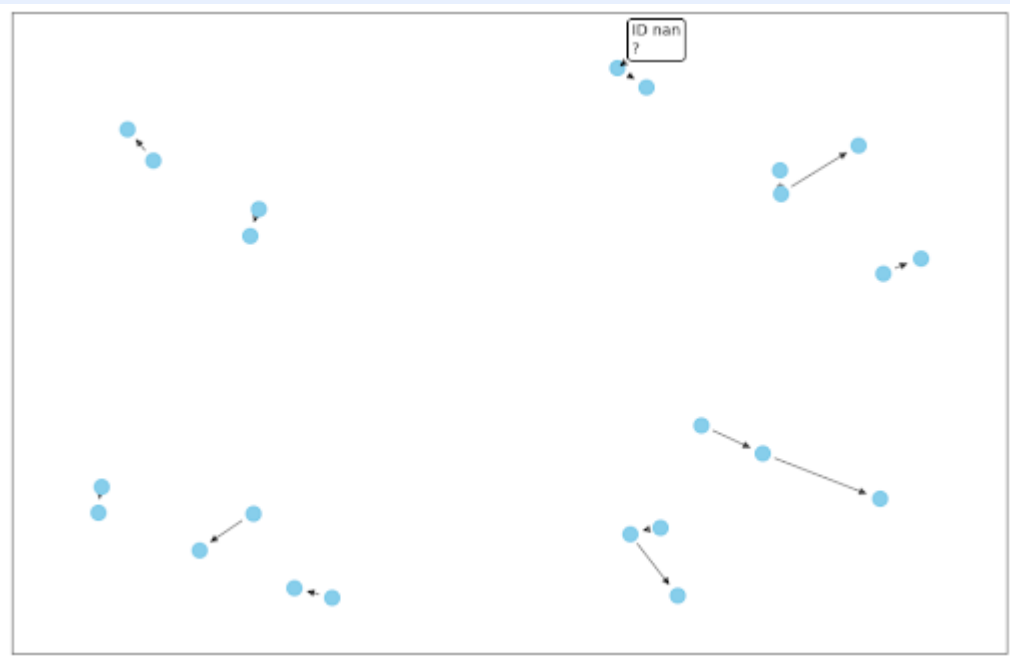
Inhalt dieses Chats

- | | |
|--|----|
| 1. Anstatt 20 weiterzusuchen und in der duckduckgo se... | 4 |
| ↳ KI-Antwort | 6 |
| 2.  Dateiprompt | 9 |
| ↳ KI-Antwort | 9 |
| 3. Implementiere diese Lösung in den folgenden Code:... | 13 |
| ↳ KI-Antwort | 48 |

TURN #1

chat_1_turn_1

USER

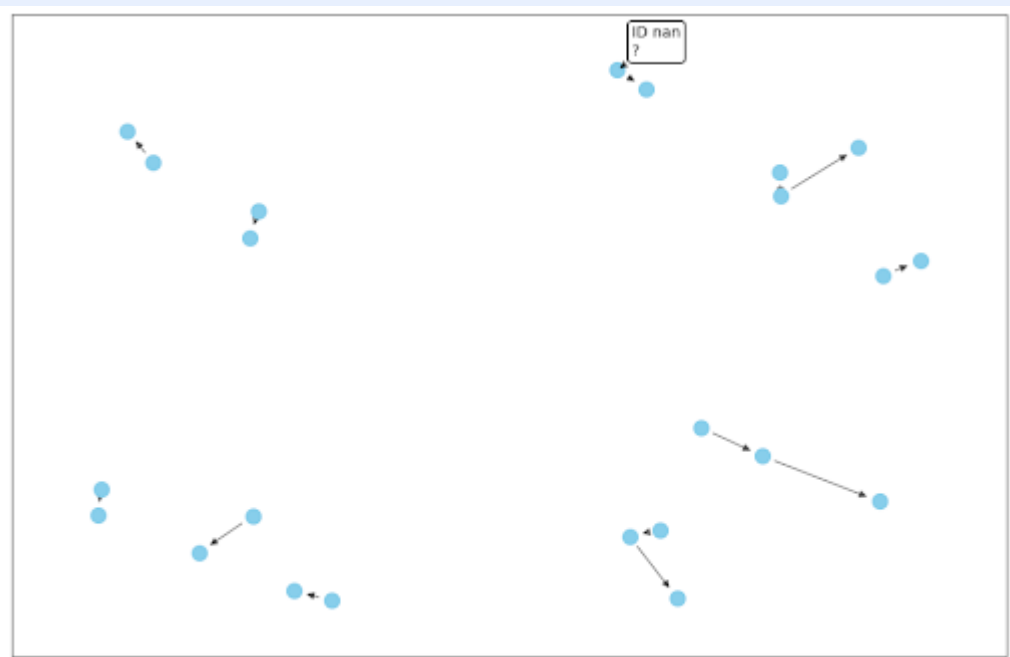


Anstatt 20 weiterzusuchen und in der duckduckgo seite zu blättern, hat das Smart Crawl nach 7 validen ergebnissen aufgehört zu suchen.

Bei seiten mit viel javascript und asynchronem inhalt ist der Crawler ins stocken gekommen.

Die Tabelle soll sortierbar sein und nach ID URL Title sortiert werden können.

Das Indizieren von unterseiten war nicht besonders erfolgreich.



MODEL

Danke für das Update. Das Bild mit dem Label ID nan ? und deine Beschreibung deuten darauf hin, dass hier mehrere Probleme (Logik, Rendering und Datenintegrität) zusammenkommen.

Hier ist eine Analyse der Probleme und konkrete Lösungsansätze, wie wir deinen "Smart Crawl" stabilisieren:

1. Das Daten-Problem: ID nan & Sortierung

Das Bild zeigt deutlich: **Du hast ein Datenintegritätsproblem.** Bevor wir die Tabelle sortierbar machen, müssen wir sicherstellen, dass die Daten valide sind. Eine Sortierung nach "ID" wird fehlschlagen oder crashen, wenn die ID NaN (Not a Number) oder null ist.

- **Ursache:** Wahrscheinlich wird die ID erst zugewiesen, *nachdem* der Crawl erfolgreich war. Wenn der Crawl bei JS-lastigen Seiten scheitert (siehe Punkt 3), wird zwar ein Eintrag angelegt, aber keine ID generiert.
- **Lösung:**
- **ID-Generierung vorziehen:** Generiere eine UUID oder einen Hash (z.B. aus der URL) *bevor* der Request startet. So hat jeder Eintrag zumindest eine ID.
- **Frontend Sortierung:** Für die Tabelle empfehle ich Standard-Bibliotheken (wie DataTables für jQuery, oder die built-in Sortierfunktionen von React/Vue), aber erst *nachdem* die NaN-Werte bereinigt sind.

2. Abbruch nach 7 Ergebnissen (DuckDuckGo Pagination)

Dass der Crawler aufhört, obwohl er 20 Ergebnisse holen soll, liegt meist an der Abbruchbedingung in der Schleife oder an Anti-Bot-Maßnahmen.

- **Logik-Check:** Prüfe deine while-Schleife.
- *Falsch:* `while i < 20` (Zählt Versuche, nicht Treffer).
- *Richtig:* `while valid_results.length < 20`.
- **DuckDuckGo Spezifikum:** DDG lädt Ergebnisse oft via "Infinite Scroll" oder einem "Mehr laden" Button nach, der per XHR (AJAX) getriggert wird. Wenn dein Crawler nur das initiale HTML parst, sieht er die Buttons für Seite 2, 3 etc. nicht oder kann sie nicht ausführen.

- **Lösung:** Du musst den "Next"-Token aus dem HTML extrahieren oder (besser) einen Browser simulieren, der scrollt.

3. JavaScript & Asynchroner Inhalt (Stocken)

Du schreibst, der Crawler stockt bei viel JS. Das ist typisch für statische Request-Libraries (wie `requests` in Python oder `axios` in Node). Sie sehen nur den leeren Container, bevor React/Angular/Vue den Inhalt rendert.

- **Die Lösung:** Du brauchst einen **Headless Browser**.
- **Empfehlung:** Nutze **Playwright** (oder Puppeteer).
- *Warum?* Playwright wartet automatisch auf `networkidle` (bis der Netzwerkverkehr sich beruhigt hat) oder bis bestimmte Selektoren sichtbar sind (`await page.waitForSelector('.content')`). Das löst das Problem mit leeren Seiten bei asynchronem Content.

4. Subseiten-Indizierung (Link-Tiefe)

Wenn Unterseiten nicht gefunden werden, liegt das oft an der URL-Normalisierung.

- **Problem:** Viele Links im HTML sind relativ (`/about` statt `https://example.com/about`).
- **Lösung:**
- Nutze eine URL-Join-Funktion, um relative Links in absolute umzuwandeln.
- **Filterung:** Ignoriere Links, die auf `#`, `mailto:`, oder Dateien (`.pdf`, `.jpg`) zeigen, um Ressourcen zu sparen.

Gezielte Fragen zur Umsetzung

Um dir den passenden Code-Fix zu geben, muss ich kurz wissen:

1. **Tech Stack:** Verwendest du Python (BeautifulSoup/Scrapy/Selenium) oder Node.js (Puppeteer/Playwright)?
2. **Datenhaltung:** Woher kommt die ID aktuell? Aus einer Datenbank (Auto-Increment) oder wird sie im Code berechnet?
3. **Visualisierung:** Womit wurde der Graph im Bild erstellt? (Scheint Matplotlib oder NetworkX zu sein – eventuell können wir die NaN Knoten dort direkt rot färben zur Diagnose).

Nächster Schritt:

Möchtest du, dass ich dir ein Snippet für **Playwright** erstelle, das speziell darauf ausgelegt ist, auf asynchrones Nachladen (Infinite Scroll) bei DuckDuckGo zu warten?

TURN #2

chat_1_turn_2

USER

id	row_id	parent_id	url	title	description	raw_content	escaped_content	depth_level	search_rank	is_book_end	created_at
1	1	NULL	https://musiclief.net/	Michael Rüting - musiclief.net	Fachinformatiker, IT-Spezialist & DJ	Kontaktieren - Hochschule - michael.rueping@fhn-net...	Kontaktieren - Hochschule - michael.rueping@fhn-net...	0	4	0	2025-12-08 02:20:09
2	1	NULL	https://cut3d.com/en/users/musiclief	All the 3D models of musiclief - Cut3	Repository of 3D printable designs shared by musiclief	- 2 Designs - 69 Downloads - 1 Follower IT pro by...	- 2 Designs - 69 Downloads - 1 Follower IT pro by...	0	5	0	2025-12-08 02:20:13
3	1	NULL	https://github.com/musiclief	musiclief - GitHub	musiclief has one repository available. Follow th...	Popular repositories Loading - KI-ITSM-Priority KI...	Popular repositories Loading - KI-ITSM-Priority KI...	0	6	0	2025-12-08 02:20:11
4	1	NULL	https://www.twitch.tv/musiclief	musiclief - Twitch				0	7	0	2025-12-08 02:20:42
5	1	NULL	https://www.mixcloud.com/musiclief/musiclief_net_24_19948_jan11_Mixcloud	musiclief_net2023-06-24_19948_jan11 - Mixcloud	Listen to the best DJs and radio presenters in the...			0	9	0	2025-12-08 02:20:48
6	4	NULL	https://musiclief.net/	Michael Rüting - musiclief.net	Fachinformatiker, IT-Spezialist & DJ	Kontaktieren - Hochschule - michael.rueping@fhn-net...	Kontaktieren - Hochschule - michael.rueping@fhn-net...	0	1	0	2025-12-08 02:19:27
7	4	NULL	https://www.musiclief.info/	www.musiclief.info				0	8	0	2025-12-08 02:40:05
8	5	NULL	https://cut3d.com/en/users/musiclief	All the 3D models of musiclief - Cut3	Repository of 3D printable designs shared by musiclief	- 2 Designs - 69 Downloads - 1 Follower IT pro by...	- 2 Designs - 69 Downloads - 1 Follower IT pro by...	0	2	1	2025-12-08 02:41:36
9	5	NULL	https://github.com/musiclief	musiclief - GitHub	musiclief has one repository available. Follow th...	Popular repositories Loading - KI-ITSM-Priority KI...	Popular repositories Loading - KI-ITSM-Priority KI...	0	3	1	2025-12-08 02:41:37
10	5	NULL	https://musiclief.net/	Michael Rüting - musiclief.net	Fachinformatiker, IT-Spezialist & DJ	Kontaktieren - Hochschule - michael.rueping@fhn-net...	Kontaktieren - Hochschule - michael.rueping@fhn-net...	0	8	1	2025-12-08 02:41:42
11	5	8	https://cut3d.com/en/users/musiclief/3d-models/...	Link	Repository of 3D printable designs shared by musiclief	- 2 Designs - 69 Downloads - 1 Follower IT pro by...	- 2 Designs - 69 Downloads - 1 Follower IT pro by...	1	2	0	2025-12-08 02:41:43
12	5	9	https://github.com/musiclief/about-content	Link	musiclief has one repository available. Follow th...	Popular repositories Loading - KI-ITSM-Priority KI...	Popular repositories Loading - KI-ITSM-Priority KI...	1	3	0	2025-12-08 02:42:06

id	row_id	parent_id	url	title	description	raw_content	escaped_content	depth_level	search_rank	is_book_end	created_at
1	1	NULL	https://musiclief.net/	Michael Rüting - musiclief.net	Fachinformatiker, IT-Spezialist & DJ	Kontaktieren - Hochschule - michael.rueping@fhn-net...	Kontaktieren - Hochschule - michael.rueping@fhn-net...	0	4	0	2025-12-08 02:20:09
2	1	NULL	https://cut3d.com/en/users/musiclief	All the 3D models of musiclief - Cut3	Repository of 3D printable designs shared by musiclief	- 2 Designs - 69 Downloads - 1 Follower IT pro by...	- 2 Designs - 69 Downloads - 1 Follower IT pro by...	0	5	0	2025-12-08 02:20:13
3	1	NULL	https://github.com/musiclief	musiclief - GitHub	musiclief has one repository available. Follow th...	Popular repositories Loading - KI-ITSM-Priority KI...	Popular repositories Loading - KI-ITSM-Priority KI...	0	6	0	2025-12-08 02:20:11
4	1	NULL	https://www.twitch.tv/musiclief	musiclief - Twitch				0	7	0	2025-12-08 02:20:42
5	1	NULL	https://www.mixcloud.com/musiclief/musiclief_net_24_19948_jan11_Mixcloud	musiclief_net2023-06-24_19948_jan11 - Mixcloud	Listen to the best DJs and radio presenters in the...			0	9	0	2025-12-08 02:20:48
6	4	NULL	https://musiclief.net/	Michael Rüting - musiclief.net	Fachinformatiker, IT-Spezialist & DJ	Kontaktieren - Hochschule - michael.rueping@fhn-net...	Kontaktieren - Hochschule - michael.rueping@fhn-net...	0	1	0	2025-12-08 02:19:27
7	4	NULL	https://www.musiclief.info/	www.musiclief.info				0	8	0	2025-12-08 02:40:05
8	5	NULL	https://cut3d.com/en/users/musiclief	All the 3D models of musiclief - Cut3	Repository of 3D printable designs shared by musiclief	- 2 Designs - 69 Downloads - 1 Follower IT pro by...	- 2 Designs - 69 Downloads - 1 Follower IT pro by...	0	2	1	2025-12-08 02:41:36
9	5	NULL	https://github.com/musiclief	musiclief - GitHub	musiclief has one repository available. Follow th...	Popular repositories Loading - KI-ITSM-Priority KI...	Popular repositories Loading - KI-ITSM-Priority KI...	0	3	1	2025-12-08 02:41:37
10	5	NULL	https://musiclief.net/	Michael Rüting - musiclief.net	Fachinformatiker, IT-Spezialist & DJ	Kontaktieren - Hochschule - michael.rueping@fhn-net...	Kontaktieren - Hochschule - michael.rueping@fhn-net...	0	8	1	2025-12-08 02:41:42
11	5	8	https://cut3d.com/en/users/musiclief/3d-models/...	Link	Repository of 3D printable designs shared by musiclief	- 2 Designs - 69 Downloads - 1 Follower IT pro by...	- 2 Designs - 69 Downloads - 1 Follower IT pro by...	1	2	0	2025-12-08 02:41:43
12	5	9	https://github.com/musiclief/about-content	Link	musiclief has one repository available. Follow th...	Popular repositories Loading - KI-ITSM-Priority KI...	Popular repositories Loading - KI-ITSM-Priority KI...	1	3	0	2025-12-08 02:42:06

MODEL

Das neue Bild der Datenbank-Tabelle ist extrem aufschlussreich. Es bestätigt genau, wo die technischen Engpässe liegen. Hier ist die Analyse deiner Daten und der Fahrplan zur Behebung der Probleme:

1. Analyse der Datenbank-Tabelle

Die Daten zeigen drei konkrete Probleme, die deine beschriebenen Fehler ("Stocken bei JS", "schlechte Sub-Indizierung") erklären:

- **Der Beweis für das JS-Problem (Rows 3, 9, 12):**
Schau dir die Spalte `raw_content` für die GitHub-Links an. Dort steht: *"Popular repositories Loading"*.
- **Diagnose:** Dein Crawler lädt nur das initiale HTML. Da GitHub (und viele moderne Seiten) die Inhalte per JavaScript nachlädt, sieht dein Crawler nur den Lade-Spinner ("Loading"), aber nicht den eigentlichen Inhalt (die Repositories). Deshalb stockt er und findet keine relevanten Keywords.
- **Das Problem der "falschen" Unterseiten (Rows 11, 12):**
Du sagtest, das Indizieren von Unterseiten war nicht erfolgreich. Die Tabelle zeigt, warum:
- Row 11 (`.../3d-models#...`) und Row 12 (`...#start-of-content`) sind **keine echten Unterseiten**.
- **Diagnose:** Das sind sogenannte "Anchor Links" (erkennbar am #). Sie führen auf dieselbe Seite, nur an eine andere Scroll-Position. Dein Crawler hält sie für neue Seiten, lädt den gleichen Inhalt nochmal herunter und verschwendet Ressourcen ("Duplicate Content").
- **ID und Sortierung:**
Die gute Nachricht: In der Datenbank ist die `id` (Spalte 1) ein sauberer Integer (1, 2, 3...). Das NaN-Problem aus dem ersten Bild ist also **kein Datenbank-Problem**, sondern entsteht vermutlich erst, wenn du die Daten für die Graphen-Visualisierung (Python/Pandas) oder die Frontend-Tabelle lädst.

2. Die Lösungen (Schritt-für-Schritt)

Um den "Smart Crawl" zu reparieren, müssen wir an drei Schrauben drehen:

A. Umstellung auf "Headless Browser" (gegen das JS-Problem)

Damit statt "Loading" der echte Text in der Datenbank landet, musst du die Seite so rendern, wie ein Browser es tut.

- **Tool:** Nutze **Playwright** (oder Puppeteer) statt requests oder axios.
- **Logik:**

```
# Pseudo-Code Beispiel für Playwright page.goto(url) # Wichtig:  
Warten, bis das Netzwerk ruhig ist (meist sind dann alle JS-  
Daten da) page.wait_for_load_state("networkidle") content =  
page.content()
```

Das löst das GitHub-Problem sofort.

B. URL-Bereinigung (gegen das Subseiten-Problem)

Bevor du eine URL in die Queue zum Crawlern legst, musst du sie "normalisieren".

- **Die Regel:** Entferne alles nach dem #.
- **Das Ergebnis:** Aus .../page#section1 und .../page#section2 wird beides .../page. Da die URL dann schon bekannt ist, wird sie nicht doppelt gecrawlt.
- **Zusatz-Tipp:** Prüfe auch Query-Parameter (?lang=de). Oft generieren diese ebenfalls Duplikate.

C. Das "7 Ergebnisse"-Problem (DuckDuckGo Pagination)

DuckDuckGo lädt Ergebnisse dynamisch nach ("Infinite Scroll"). Ein statischer Request sieht nur die ersten paar Ergebnisse.

- **Lösung mit Playwright:**
 - Lade die DDG-Suchseite.
 - Simuliere das Scrollen ans Ende der Seite (`page.keyboard.press("End")`).
 - Warte auf den Button "Mehr laden" (Selector oft `.result--more__btn`) und klicke ihn (`page.click(...)`).
 - Erst *dann* die Links extrahieren.
-

3. Für die sortierbare Tabelle

Da die Datenbank sauber ist, kannst du die Sortierung direkt über SQL lösen, bevor die Daten ins Frontend gehen. Das verhindert NaN-Fehler in der Anzeige.

SQL Query Vorschlag (für deine Tabellenansicht):

```
SELECT id, url, title, description
FROM deine_tabelle
WHERE run_id = (SELECT MAX(run_id) FROM deine_tabelle) -- Nur den
aktuellsten Lauf zeigen
ORDER BY id ASC; -- Oder nach url/title
```

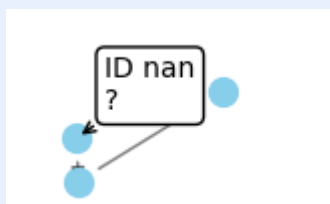
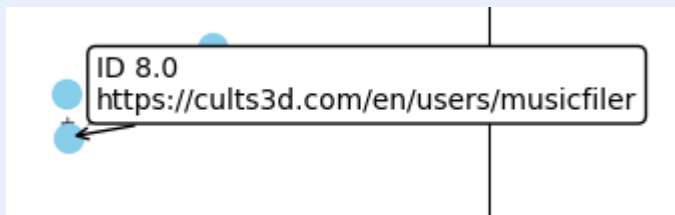
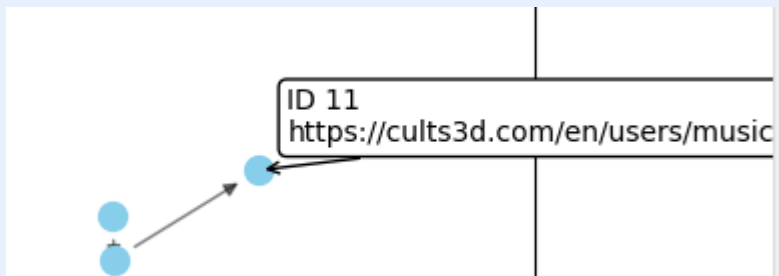
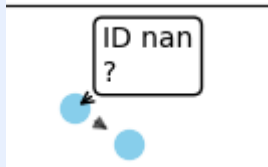
Möglicher nächster Schritt:

Soll ich dir eine **Python-Funktion (mit Playwright)** schreiben, die URLs automatisch bereinigt (das #-Problem löst) und wartet, bis der "Loading"-Text verschwunden ist?

TURN #3

chat_1_turn_3

USER



Implementiere diese Lösung in den folgenden Code:

```
import tkinter as tk
```

```
from tkinter import ttk, scrolledtext, messagebox, Menu, filedialog, Toplevel
```

```
import threading
```

```
import time
```

```
import sys
```

```
import webbrowser
```

```
import socket

import mysql.connector

from mysql.connector import errorcode

import html

import json

import os

import warnings

import re

import pandas as pd

import trafilatura

from langdetect import detect, detect_langs, LangDetectException

import networkx as nx

from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

import matplotlib.pyplot as plt

import asyncio

import base64

from datetime import datetime

from bs4 import BeautifulSoup
```

--- 1. CRITICAL SETUP & GLOBALS ---

```
sys.stdout.reconfigure(encoding='utf-8')

os.environ["PYTHONWARNINGS"] = "ignore"

warnings.simplefilter(action='ignore', category=FutureWarning)

warnings.simplefilter(action='ignore', category=UserWarning)
```

Globaler Pfad für Chrome Profil

```
CHROME_USER_DATA_DIR = os.path.join(os.getenv('LOCALAPPDATA'), 'Google', 'Chrome',  
'User Data') if os.name == 'nt' else ""
```

--- 2. IMPORTS ---

```
try:  
  
import socks  
  
except ImportError:  
  
pass  
  
try:  
  
from duckduckgo_search import DDGS  
  
except ImportError:  
  
try:  
  
from ddgs import DDGS  
  
except ImportError:  
  
pass  
  
from deep_translator import GoogleTranslator  
  
from playwright.sync_api import sync_playwright  
  
CRAWL4AI_AVAILABLE = False  
  
try:  
  
from crawl4ai import AsyncWebCrawler, BrowserConfig, CrawlerRunConfig, CacheMode  
  
CRAWL4AI_AVAILABLE = True  
  
except ImportError:  
  
pass
```

3. KONFIGURATION

```
CONFIG_FILE = "crawler_config.json"

DEFAULT_CONFIG = {

    "db_host": "localhost", "db_port": "3306",

    "db_user": "root", "db_password": "",

    "db_name": "search_index_v3", "tor_port": "9050"

}

REGION_MAPPING = {

    'de': {'ddg': 'de-de', 'locale': 'de-DE', 'header': 'de-DE,de;q=0.9,en-US;q=0.8,en;q=0.7'},

    'en': {'ddg': 'us-en', 'locale': 'en-US', 'header': 'en-US,en;q=0.9'},

    'fr': {'ddg': 'fr-fr', 'locale': 'fr-FR', 'header': 'fr-FR,fr;q=0.9,en;q=0.8'},

    'es': {'ddg': 'es-es', 'locale': 'es-ES', 'header': 'es-ES,es;q=0.9,en;q=0.8'},

    'it': {'ddg': 'it-it', 'locale': 'it-IT', 'header': 'it-IT,it;q=0.9,en;q=0.8'},

    'cn': {'ddg': 'cn-zh', 'locale': 'zh-CN', 'header': 'zh-CN,zh;q=0.9,en;q=0.8'}

}

DEFAULT_REGION_CFG = {'ddg': 'wt-wt', 'locale': 'en-US', 'header': 'en-US,en;q=0.9'}

DOMAIN_BLACKLIST = [

    "zhihu.com", "bilibili.com", "facebook.com", "twitter.com", "instagram.com",

    "youtube.com", "tiktok.com", "linkedin.com", "google.com", "duckduckgo.com",

    "bing.com", "yahoo.com", "baidu.com", "yandex.com"

]
```

```
JUNK_PATTERNS = [  
    r'/login', r'/signin', r'/signup', r'/register', r'/auth',  
    r'403 forbidden', r'access denied', r'captcha', r'verify you are a human',  
    r'security check', r'cloudflare', r'{"message":', r'password',  
    r'forgot-password', r'reset-password'  
]
```

4. UTILS & QUALITY GATE

```
class ToolTip:  
    def __init__(self, widget, text):  
        self.widget = widget;  
        self.text = text;  
        self.tip_window = None  
        widget.bind("", self.show_tip);  
        widget.bind("", self.hide_tip)  
        def show_tip(self, event=None):  
            if self.tip_window or not self.text: return  
            x, y, _, _ = self.widget.bbox("insert")  
            x += self.widget.winfo_rootx() + 25;  
            y += self.widget.winfo_rooty() + 25  
            self.tip_window = tw = Toplevel(self.widget);  
            tw.wm_overrideredirect(True);
```

```
tw.wm_geometry(f"+{x}+{y}")

tk.Label(tw, text=self.text, justify=tk.LEFT, background="#ffffe0", relief=tk.SOLID,
borderwidth=1,

font=("tahoma", "8", "normal")).pack(ipadx=1)

def hide_tip(self, event=None):

if self.tip_window: self.tip_window.destroy(); self.tip_window = None

def load_config():

if os.path.exists(CONFIG_FILE):

try:

return json.load(open(CONFIG_FILE, 'r'))

except:

return DEFAULT_CONFIG

return DEFAULT_CONFIG

def save_config_to_file(data): json.dump(data, open(CONFIG_FILE, 'w'), indent=4)

def setup_proxy(use_tor, port, log_func):

try:

if use_tor:

socks.set_default_proxy(socks.SOCKS5, "localhost", port);

socket.socket = socks.socksocket

log_func(f"[NET] Tor Proxy (:{port}) aktiv.")

else:

socks.set_default_proxy();

socket.socket = socket._socketobject if hasattr(socket, "_socketobject") else socket.socket

log_func("[NET] Direkte Verbindung.")

except Exception as e:

log_func(f"[NET ERR] {e}")
```

```
class QualityGate:

    @staticmethod
    def is_url_allowed(url):
        if not url or not url.startswith("http"): return False

        url_lower = url.lower()

        for d in DOMAIN_BLACKLIST:
            if d in url_lower: return False

        for p in JUNK_PATTERNS:
            if p.startswith("/") and p in url_lower: return False

        return True

    @staticmethod
    def is_junk_content(title, content):
        combined = (title + " " + content).lower()[:1500]

        for p in JUNK_PATTERNS:
            if p.replace("/", "") in combined: return True

        return False

    @staticmethod
    def contains_keyword(content, keyword):
        if not keyword or not content: return True

        return keyword.lower() in content.lower()

    @staticmethod
    def check_language(text, html_raw, target_lang):
```

1. HTML Meta Check

```
if html_raw:
```

```
try:
```

```
soup = BeautifulSoup(html_raw, 'html.parser')

html_tag = soup.find('html')

if html_tag and html_tag.has_attr('lang'):

    if html_tag['lang'].lower().startswith(target_lang): return True, "Meta-Tag Match"

except:

    pass
```

2. Text Analyse

```
if len(text) < 150: return True, "Short (Pass)"

try:

    langs = detect_langs(text)

    if langs[0].lang.startswith(target_lang): return True, f"Text ({langs[0].prob:.2f})"

    if target_lang in ['de', 'en', 'fr', 'es'] and langs[0].lang in ['ru', 'zh-cn', 'ja', 'ar']:

        return False, f"Wrong Lang: {langs[0].lang}"

except:

    pass

return True, "Fallback"
```

5. DATABASE MANAGER

```
class DatabaseManager:

    def __init__(self, logger_func, config):
```

```
self.log = logger_func;

self.config = config;

self.conn = None;

self.cursor = None

def connect(self):

    try:

        self.conn = mysql.connector.connect(

            user=self.config['db_user'], password=self.config['db_password'],

            host=self.config['db_host'], port=int(self.config['db_port']),

            database=self.config['db_name'], raise_on_warnings=True

        )

        self.cursor = self.conn.cursor();

        self.create_tables();

        return True

    except mysql.connector.Error as err:

        if err.errno == errorcode.ER_BAD_DB_ERROR:

            if self._create_db(): return self.connect()

            if self.log: self.log(f"[DB ERR] {err}")

        return False

    def _create_db(self):

        try:

            c = mysql.connector.connect(user=self.config['db_user'],

                password=self.config['db_password'],

                host=self.config['db_host'], port=int(self.config['db_port']))

            c.cursor().execute(f"CREATE DATABASE IF NOT EXISTS {self.config['db_name']} DEFAULT

                CHARACTER SET utf8mb4")

            c.close();
```

```
return True

except:

return False

def get_conn_pandas(self):

try:

return mysql.connector.connect(user=self.config['db_user'],
password=self.config['db_password'],

host=self.config['db_host'], port=int(self.config['db_port']),

database=self.config['db_name'])

except:

return None

def create_tables(self):

try:

self.cursor.execute(

"CREATE TABLE IF NOT EXISTS search_runs (id INT AUTO_INCREMENT PRIMARY KEY, query
VARCHAR(255), lang VARCHAR(10), mode VARCHAR(50), created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP)")

self.cursor.execute("""

CREATE TABLE IF NOT EXISTS crawled_data

(

id

INT

AUTO_INCREMENT

PRIMARY

KEY,

run_id

INT,
```



```
res = self.cursor.fetchone()

return res[0] if res else None

except:

return None

def save_node(self, run_id, parent_id, url, title, desc, content, depth, rank, is_list=False):

try:

eid = self.url_exists(url, run_id)

if eid: return eid, True

esc = html.escape(content) if content else ""

self.cursor.execute("""

INSERT INTO crawled_data (run_id, parent_id, url, title, description, raw_content,

escaped_content, depth_level, search_rank, is_link_list)

VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)""",

(run_id, parent_id, url, title, desc, content, esc, depth, rank, is_list))

self.conn.commit();

return self.cursor.lastrowid, False

except:

return None, False

def close(self):

if self.conn: self.conn.close()
```

6. UNIVERSAL CRAWLER WORKER (FIXED SYNTAX)

```
class UniversalCrawlerWorker:

    def __init__(self, cfg, log):

        self.cfg = cfg; self.log = log

    def run(self, params):

        query = params['query'];

        source = params['source'];

        langs = params['langs']

        use_tor = params['tor'];

        visible = params['visible'];

        trans = params['trans']

        smart_filter = params['smart'];

        strict_keyword = params['strict']

        mode_depth = params['mode_depth'];

        recursive = params['recursive'];

        target_valid = params['target_valid']

        tor_port = int(self.cfg.get("tor_port", 9050))

        setup_proxy(use_tor, tor_port, self.log)

        db = DatabaseManager(self.log, self.cfg)
```

```
if not db.connect(): return

with sync_playwright() as p:

    proxy = {"server": f"socks5://localhost:{tor_port}"} if use_tor else None

    browser = p.chromium.launch(headless=not visible, proxy=proxy,

    args=["--start-maximized", "--disable-blink-features=AutomationControlled"])

    loop_langs = langs if source == "search" else ["direct"]

    for lang in loop_langs:

        l_cfg = REGION_MAPPING.get(lang)

        if not isinstance(l_cfg, dict): l_cfg = DEFAULT_REGION_CFG

        context = browser.new_context(

        viewport={"width": 1280, "height": 800},

        locale=l_cfg['locale'], timezone_id='UTC',

        extra_http_headers={"Accept-Language": l_cfg['header']}

        )

        context.add_init_script("Object.defineProperty(navigator, 'webdriver', {get: () =>

        undefined}))")
```

--- BLOCK MEDIA ---

```
def block_media(route):

    if route.request.resource_type in ["image", "media", "font"]:

        try:

            route.abort()

        except:

            pass

        else:

            try:
```

```
route.continue_()

except:

    pass

context.route("**/*", block_media)

queue_items = []
```

PHASE 1: QUEUE FILL

```
if source == "search":

    self.log(f"--- SUCHE {lang.upper()} (Ziel: {target_valid}) ---")

    term = query

    if trans and lang != 'de':

        try:

            term = GoogleTranslator(source='auto', target=lang).translate(query)

        except:

            pass

        try:

            db.cursor.execute("INSERT INTO search_runs (query, lang, mode) VALUES (%s, %s, %s)",

                               (term, lang,

                                f"mode_{mode_depth}")); run_id = db.cursor.lastrowid; db.conn.commit()

        except:

            continue

    if visible:

        self.log("[VISUAL] Starte Browser-Suche...")

        try:

            s_page = context.new_page()

            ddg_url = f"https://duckduckgo.com/?q={term}&kl={l_cfg['ddg']}&t=h_"
```

```
s_page.goto(ddg_url)
```

Warte auf Titel-Links

```
try:
```

```
s_page.wait_for_selector("a[data-testid='result-title-a']", timeout=10000)
```

```
except:
```

```
self.log(" -> Timeout bei DuckDuckGo Load")
```

```
links = s_page.locator("a[data-testid='result-title-a']").all()
```

```
cnt = 0
```

```
for link in links:
```

```
if cnt >= target_valid * 3: break
```

```
try:
```

```
href = link.get_attribute("href")
```

```
txt = link.inner_text()
```

```
if href and QualityGate.is_url_allowed(href):
```

```
queue_items.append((href, txt, cnt + 1, 0, None))
```

```
cnt += 1
```

```
except:
```

```
pass
```

```
s_page.close()
```

```
self.log(f" -> {len(queue_items)} Roh-Links gefunden.")
```

```
except Exception as e:
```

```
self.log(f"[VISUAL ERR] {e}")
```

```
else:
```

```
try:
```

```
with DDGS() as ddgs:
```

```
res = list(

ddgs.text(term, region=l_cfg.get('ddg', 'wt-wt'), max_results=target_valid * 3,
backend="html"))

for idx, r in enumerate(res):

if QualityGate.is_url_allowed(r['href']):

queue_items.append((r['href'], r['title'], idx + 1, 0, None))

except Exception as e:

self.log(f"[API ERR] {e}")

else:

self.log(f"--- DIREKT ---")

urls = get_urls_from_input(query, self.log)

if not urls: break

try:

db.cursor.execute("INSERT INTO search_runs (query, lang, mode) VALUES (%s, 'raw', 'direct'",
(os.path.basename(query)[

:50],)); run_id = db.cursor.lastrowid; db.conn.commit()

except:

continue

for i, u in enumerate(urls): queue_items.append((u, "Start URL", i + 1, 0, None))
```

PHASE 2: CRAWLING

```
visited = set()

valid_cnt = 0

while queue_items:

if source == "search" and valid_cnt >= target_valid:

self.log(f"*** ZIEL ERREICHT ***");
```

```
break

u, t, r, d, pid = queue_items.pop(0)

if u in visited: continue

visited.add(u)

if db.url_exists(u, run_id):

    self.log(f" -> [SKIP] DB: {u[:30]}...")

    valid_cnt += 1;

    continue

self.log(f"[NODE D{d}] {u[:40]}...")

content = "";

desc = "";

child_links = []

should_visit = (mode_depth >= 1) or (d > 0)

if should_visit:

    try:

        page = context.new_page()

        page.set_extra_http_headers({"Accept-Language": l_cfg['header']})

    try:

        page.goto(u, timeout=35000, wait_until="domcontentloaded")

    except:

        try:

            page.close()

        except:

            pass

        continue

    p_content = page.content()
```

```
p_text = page.evaluate("document.body.innerText")

p_title = page.title()

if QualityGate.is_junk_content(p_title, p_text):

    self.log(" -> [DROP] Junk/Login")

    page.close()

    continue

if source == "search" and smart_filter:

    ok, msg = QualityGate.check_language(p_text, p_content, lang)

    if not ok:

        self.log(f" -> [DROP] {msg}")

        page.close()

        continue

if source == "search" and strict_keyword and query:

    if not QualityGate.contains_keyword(p_text, query):

        self.log(" -> [DROP] Keyword fehlt.")

        page.close()

        continue

if mode_depth >= 1: desc = page.locator('meta[name="description"]').get_attribute(
    "content") or ""

if mode_depth >= 2: content = trafilatura.extract(p_content) or ""

max_tree = 1 if mode_depth == 3 else 2 if mode_depth == 4 else 0

limit = 3 if recursive else max_tree

if d < limit:

    hrefs = page.evaluate("Array.from(document.links).map(a => a.href)")

    for h in hrefs:

        if h.startswith("http") and QualityGate.is_url_allowed(h): child_links.append(h)
```

```
page.close()

except Exception as e:

self.log(f" -> [ERR] {e}")

try:

page.close()

except:

pass

node_id, exists = db.save_node(run_id, pid, u, t, desc, content, d, r,

is_list=(len(child_links) > 0))

if exists:

self.log(" -> [DUP]");

valid_cnt += 1

elif node_id:

self.log(f" -> [SAVED] {len(content)} Chars")

valid_cnt += 1

if child_links:

cnt = 0

for l in child_links:

if cnt < 20: queue_items.append((l, "Link", r, d + 1, node_id)); cnt += 1

context.close()

browser.close()

db.close();

self.log("--- FINISHED ---")
```

GUI COMPONENTEN

```
class UniversalCrawlerTab(ttk.Frame):

    def __init__(self, parent, app):

        super().__init__(parent);

        self.app = app;

        self.setup_ui()

    def setup_ui(self):

        pnl = ttk.LabelFrame(self, text="Konfiguration", padding=10);

        pnl.pack(fill="x", padx=10, pady=5)

        self.v_src = tk.StringVar(value="search")

        r1 = ttk.Radiobutton(pnl, text="🔍 Suche", variable=self.v_src, value="search",
                             command=self.upd);

        r1.grid(row=0, column=0)

        Tooltip(r1, "Nutzt DuckDuckGo (API oder Browser) um URLs zu finden.")

        r2 = ttk.Radiobutton(pnl, text="📁 Direkt/Datei", variable=self.v_src, value="direct",
                             command=self.upd);

        r2.grid(row=0, column=1)

        Tooltip(r2, "Crawlt eine spezifische URL oder eine Liste aus einer .txt Datei.")

        ttk.Label(pnl, text="Input:").grid(row=0, column=2, padx=(10, 0));

        self.e_in = ttk.Entry(pnl, width=30);

        self.e_in.grid(row=0, column=3)
```

```
self.b_file = ttk.Button(pnl, text="...", width=3, command=self.pick, state="disabled");
self.b_file.grid(row=0, column=4)

ttk.Label(pnl, text="Sprachen:").grid(row=0, column=5, padx=(10, 0));

self.e_l = ttk.Entry(pnl, width=10);

self.e_l.insert(0, "de,en");

self.e_l.grid(row=0, column=6)

ToolTip(self.e_l, "Kommagetrennt (de,en,fr). Nur für Suchmodus relevant.")

frame_mode = ttk.Frame(pnl);

frame_mode.grid(row=1, columnspan=7, pady=10, sticky="w")

ttk.Label(frame_mode, text="Modus:").pack(side="left")

self.cb_mode = ttk.Combobox(frame_mode,

values=["0: Nur URLs sammeln", "1: + Meta Beschreibung", "2: + Volltext (Content)",

"3: + Linkliste (Level 1)", "4: + Deep Links (Level 2)"], width=30);

self.cb_mode.current(2);

self.cb_mode.pack(side="left", padx=5)

ToolTip(self.cb_mode,

"Bestimmt, wie tief eine gefundene Seite analysiert wird.\n3 & 4 erstellen Baumstrukturen.")

self.v_rec = tk.BooleanVar(value=False);

c_rec = ttk.Checkbutton(frame_mode, text="Rekursiv", variable=self.v_rec);

c_rec.pack(side="left", padx=10)

ToolTip(c_rec, "Wenn aktiv, werden gefundene Links erneut gescannt (entsprechend dem

Modus).")

ttk.Label(frame_mode, text="Ziel (Valid):").pack(side="left", padx=(10, 0));

self.s_valid = ttk.Spinbox(frame_mode, from_=1, to=1000, width=5);

self.s_valid.set(20);

self.s_valid.pack(side="left")
```

```
ToolTip(self.s_valid, "Crawler sucht so lange weiter, bis diese Anzahl an ECHTEN Treffern  
erreicht ist.")  
  
frame_opt = ttk.Frame(pnl);  
  
frame_opt.grid(row=2, columnspan=7, pady=5, sticky="w")  
  
self.v_vis = tk.BooleanVar(value=True);  
  
c1 = ttk.Checkbutton(frame_opt, text="Sichtbar", variable=self.v_vis);  
  
c1.pack(side="left")  
  
ToolTip(c1, "Zeigt den Browser an. Wichtig für Visuelle Suche.")  
  
self.v_tor = tk.BooleanVar(value=False);  
  
c2 = ttk.Checkbutton(frame_opt, text="Tor", variable=self.v_tor);  
  
c2.pack(side="left", padx=5)  
  
ToolTip(c2, "Leitet Traffic über Tor (SOCKS5).")  
  
self.v_tr = tk.BooleanVar(value=False);  
  
c3 = ttk.Checkbutton(frame_opt, text="Übersetzen", variable=self.v_tr);  
  
c3.pack(side="left", padx=5)  
  
ToolTip(c3, "Übersetzt den Suchbegriff automatisch.")  
  
self.v_sm = tk.BooleanVar(value=True);  
  
c4 = ttk.Checkbutton(frame_opt, text="Smart Filter", variable=self.v_sm);  
  
c4.pack(side="left", padx=5)  
  
ToolTip(c4, "Filtert falsche Sprachen und Müll-Seiten.")  
  
self.v_strict = tk.BooleanVar(value=True);  
  
c5 = ttk.Checkbutton(frame_opt, text="Zwang: Keyword", variable=self.v_strict);  
  
c5.pack(side="left", padx=5)  
  
ToolTip(c5, "Speichert NUR, wenn das Suchwort im Text der Seite vorkommt.")  
  
ttk.Button(pnl, text="🚀 STARTEN", command=self.run).grid(row=0, column=7, rowspan=3,  
padx=10, sticky="nsew")  
  
self.log_txt = scrolledtext.ScrolledText(self, height=15, bg="black", fg="lime");
```

```
self.log_txt.pack(fill="both", expand=True, padx=10)

def log(self, m):

self.log_txt.insert(tk.END, m + "\n"); self.log_txt.see(tk.END)

def pick(self):

f = filedialog.askopenfilename();

if f: self.e_in.delete(0, tk.END); self.e_in.insert(0, f)

def upd(self):

if self.v_src.get() == "direct":

self.b_file.config(state="normal"); self.e_l.config(state="disabled"); self.v_strict.set(False)

else:

self.b_file.config(state="disabled"); self.e_l.config(state="normal")

def run(self):

params = {

'query': self.e_in.get(), 'source': self.v_src.get(), 'langs': self.e_l.get().split(','),

'tor': self.v_tor.get(), 'visible': self.v_vis.get(), 'trans': self.v_tr.get(),

'smart': self.v_sm.get(), 'strict': self.v_strict.get(),

'mode_depth': int(self.cb_mode.get().split(":")[0]),

'recursive': self.v_rec.get(), 'target_valid': int(self.s_valid.get())

}

w = UniversalCrawlerWorker(self.app.app_config, self.log)

threading.Thread(target=w.run, args=(params,), daemon=True).start()

class VisuTab(ttk.Frame):

def __init__(self, parent, app):

super().__init__(parent);

self.app = app;

self.df = None;
```

```
self.setup_ui()

def setup_ui(self):
    top = ttk.Frame(self);
    top.pack(fill="x", padx=10);

    ttk.Button(top, text="Laden", command=self.load).pack(side="left")

    self.nb = ttk.Notebook(self);
    self.nb.pack(fill="both", expand=True)

    self.tab_tbl = ttk.Frame(self.nb);
    self.nb.add(self.tab_tbl, text="Tabelle")

    tbl_frame = ttk.Frame(self.tab_tbl);
    tbl_frame.pack(fill="both", expand=True)

    self.tree = ttk.Treeview(tbl_frame, columns=("ID", "PID", "URL", "Title"), show="headings");
    self.tree.pack(side="left", fill="both", expand=True)

    sc = ttk.Scrollbar(tbl_frame, orient="vertical", command=self.tree.yview);
    sc.pack(side="right", fill="y");

    self.tree.configure(yscroll=sc.set)

    for c in ("ID", "PID", "URL", "Title"): self.tree.heading(c, text=c)

    self.tree.bind("<>", self.on_tbl_sel)

    self.ctx_menu = Menu(self, tearoff=0)

    self.ctx_menu.add_command(label="Deep Crawl (mit Keyword)", command=lambda:
self.launch_deep(True))

    self.ctx_menu.add_command(label="Sitemap / Full Scan (ohne Keyword)",
command=lambda: self.launch_deep(False))

    self.tree.bind("", self.show_ctx)

    self.txt_preview = scrolledtext.ScrolledText(self.tab_tbl, height=10, bg="#eee");
    self.txt_preview.pack(fill="x", padx=5, pady=5)

    self.tab_graph = ttk.Frame(self.nb);
```

```
self.nb.add(self.tab_graph, text="Baum-Graph")

self.canv = ttk.Frame(self.tab_graph);

self.canv.pack(fill="both", expand=True)

def load(self):

    db = DatabaseManager(None, self.app.app_config);

    conn = db.get_conn_pandas()

    if not conn: return

    self.df = pd.read_sql(

        "SELECT id, parent_id, url, title, raw_content FROM crawled_data ORDER BY id DESC LIMIT
        500", conn);

    conn.close()

    for i in self.tree.get_children(): self.tree.delete(i)

    for _, r in self.df.iterrows(): self.tree.insert("", "end",

        values=(r['id'], r['parent_id'], r['url'], r['title']))

    self.draw_graph(self.df)

    def show_ctx(self, event):

        item = self.tree.identify_row(event.y)

        if item:

            self.tree.selection_set(item);

            self.ctx_menu.post(event.x_root, event.y_root)

    def launch_deep(self, use_keyword):

        sel = self.tree.selection()

        if not sel: return

        vals = self.tree.item(sel[0])['values']

        url = vals[2]

        crawler_tab = self.app.tabs_map["crawler"]
```

```
crawler_tab.e_in.delete(0, tk.END);

crawler_tab.e_in.insert(0, url)

crawler_tab.v_src.set("direct");

crawler_tab.upd()

crawler_tab.cb_mode.current(4)

crawler_tab.v_rec.set(True)

crawler_tab.v_strict.set(use_keyword)

self.app.nb.select(crawler_tab)

messagebox.showinfo("Deep Crawl", f"URL übernommen: {url}\nModus wurde eingestellt.
\nBitte 'STARTEN' klicken.")

def on_tbl_sel(self, e):

    sel = self.tree.selection()

    if not sel: return

    iid = self.tree.item(sel[0])['values'][0]

    row = self.df[self.df['id'] == iid].iloc[0]

    self.txt_preview.delete("1.0", tk.END);

    self.txt_preview.insert(tk.END, row['raw_content'] or "")

    def draw_graph(self, df):

        for w in self.canv.winfo_children(): w.destroy()

        G = nx.DiGraph();

        node_map = {}

        for _, r in df.iterrows():

            G.add_node(r['id']);

            node_map[r['id']] = r['url']

            if r['parent_id']: G.add_edge(r['parent_id'], r['id'])

        fig, ax = plt.subplots(figsize=(6, 5));
```

```
pos = nx.spring_layout(G, k=0.2)

nodes = nx.draw_networkx_nodes(G, pos, ax=ax, node_size=100, node_color='skyblue')

nx.draw_networkx_edges(G, pos, ax=ax, alpha=0.5, arrows=True)

annot = ax.annotate("", xy=(0, 0), xytext=(10, 10), textcoords="offset points",
bbox=dict(boxstyle="round", fc="w"), arrowprops=dict(arrowstyle="->"));

annot.set_visible(False)

def update_annot(ind):

    idx = ind["ind"][0];

    nid = list(G.nodes())[idx];

    url = node_map.get(nid, "?")

    annot.xy = pos[nid];

    annot.set_text(f"ID {nid}\n{url}")

def hover(event):

    vis = annot.get_visible()

    if event.inaxes == ax:

        cont, ind = nodes.contains(event)

        if cont:

            update_annot(ind); annot.set_visible(True); fig.canvas.draw_idle()

        else:

            if vis: annot.set_visible(False); fig.canvas.draw_idle()

    fig.canvas.mpl_connect("motion_notify_event", hover)

c = FigureCanvasTkAgg(fig, master=self.canv);

c.draw();

c.get_tk_widget().pack(fill="both", expand=True)

class GeminiTab(ttk.Frame):

    def __init__(self, parent, app):
```

```
super().__init__(parent);

self.app = app;

self.setup_ui()

def setup_ui(self):

if not CRAWL4AI_AVAILABLE:

    ttk.Label(self, text="Crawl4AI fehlt.", fg="red").pack(pady=20)

return

lbl = tk.Label(self, text="Gemini URLs:", font=("Segoe UI", 11, "bold"));

lbl.pack(anchor="w", padx=20, pady=(20, 5))

self.url_input = scrolledtext.ScrolledText(self, height=10);

self.url_input.pack(fill="x", padx=20)

self.headless_var = tk.BooleanVar(value=False);

tk.Checkbutton(self, text="Headless", variable=self.headless_var).pack(anchor="w", padx=20)

btn = tk.Button(self, text="START GEMINI", command=self.start_thread, bg="#d32f2f",
fg="white", height=2);

btn.pack(fill="x", padx=20, pady=20)

self.log_area = scrolledtext.ScrolledText(self, height=12, bg="#1e1e1e", fg="#00ff00");

self.log_area.pack(fill="x", side="bottom", padx=10, pady=10)

if os.path.exists("urls.txt"):

    with open("urls.txt", "r", encoding="utf-8") as f: self.url_input.insert("1.0", f.read())

def log(self, msg):

    self.log_area.insert(tk.END, f"[{datetime.now().strftime('%H:%M:%S')}] {msg}\n");
    self.log_area.see(tk.END)

def start_thread(self):

    urls = self.url_input.get("1.0", tk.END).strip().splitlines()

    urls = [u.strip() for u in urls if "gemini.google.com" in u]

    if not urls: messagebox.showerror("Fehler", "Keine URLs."); return
```

```
threading.Thread(target=self.run_async, args=(urls,), daemon=True).start()

def run_async(self, urls):
    try:
        asyncio.run(self.crawl_process(urls))
    except Exception as e:
        self.log(f"Error: {e}")

    async def crawl_process(self, urls):
        self.log("Starte Crawl4AI...")

        js_scroll = "window.scrollTo(0, document.body.scrollHeight); return document.body.scrollHeight;"

        out = "crawl_results";
        os.makedirs(out, exist_ok=True);

        sc_dir = os.path.join(out, "screenshots");
        os.makedirs(sc_dir, exist_ok=True)

        b_cfg = BrowserConfig(verbose=True, headless=self.headless_var.get(),
                               user_data_dir=CHROME_USER_DATA_DIR,
                               use_managed_browser=True)

        r_cfg = CrawlerRunConfig(cache_mode=CacheMode.BYPASS, js_code=js_scroll,
                                  wait_for="share-viewer",

                                  screenshot=True, magic=True)

        data = []

        async with AsyncWebCrawler(config=b_cfg) as crawler:
            for i, url in enumerate(urls):
                self.log(f"Lade {url}...")

                res = await crawler.arun(url=url, config=r_cfg)

                if not res.success: self.log(f"Fehler: {res.error_message}"); continue

                spath = None
```

```
if res.screenshot:

    sdata = base64.b64decode(res.screenshot)

    fname = f"chat_{int(datetime.now().timestamp())}_{i}.png";

    spath = os.path.join(sc_dir, fname)

    with open(spath, "wb") as f: f.write(sdata)

    cdata = self.parse_html(res.html, url);

    cdata['screenshot_path'] = spath

    data.append(cdata);

    self.log(f"OK: {len(cdata['turns'])} Turns.")

    await asyncio.sleep(2)

    self.app.root.after(0, lambda: ReviewWindow(self.app.root, data, self.save_report))

    def clean_html(self, el):

        if not el: return ""

        for s in ["script", "iframe", "noscript", "svg", "style"]:

            for t in el.select(s): t.decompose()

        return el.decode_contents()

    def parse_html(self, html_c, url):

        soup = BeautifulSoup(html_c, 'html.parser')

        main = soup.find('share-viewer') or soup.find('main') or soup.body

        date = datetime.now().strftime('%d.%m.%Y');

        turns = []

        if main:

            processed = set();

            block = []

            for el in main.find_all(True):

                if el in processed: continue
```

```
cls = el.get("class", []) or []

if 'user-query-bubble-with-background' in cls or el.name == 'user-query-container':

if block: turns.append({"role": "USER", "html": f"

    {chr(10).join(block)}

",

"text": chr(10).join(block)}); block = []

processed.add(el);

[processed.add(c) for c in el.find_all(True)]

turns.append({"role": "USER", "html": self.clean_html(el), "text": el.get_text()})

elif 'query-text-line' in cls:

processed.add(el);

block.append(el.get_text())

elif 'message-content' in cls or (

el.has_attr('data-test-id') and el['data-test-id'] == 'model-response'):

if block: turns.append({"role": "USER", "html": f"

    {chr(10).join(block)}

",

"text": chr(10).join(block)}); block = []

processed.add(el);

[processed.add(c) for c in el.find_all(True)]

turns.append({"role": "MODEL", "html": self.clean_html(el), "text": el.get_text()})

if block: turns.append(

{"role": "USER", "html": f"

    {chr(10).join(block)}

", "text": chr(10).join(block)})
```

```
t = "Chat Export";

h1 = soup.find('h1')

if h1: t = h1.get_text().strip()

return {"title": t, "url": url, "chat_date": date, "turns": turns}

def save_report(self, data):

    p = filedialog.asksaveasfilename(defaulttextextension=".html", filetypes=[("HTML", "*.html")])

    if not p: return

    css = "body{font-family:sans-serif;padding:
20px;background:#eee}.chat{background:white;padding:20px;margin-bottom:20px;border-
radius:8px}.role-USER{background:#e3f2fd;padding:15px;margin:10px 0;border-left:5px solid
#2196f3}.role-MODEL{background:#fff;padding:15px;margin:10px 0;border-left:5px solid
#4caf50}img.screenshot{max-width:100%;border:1px solid #ccc;margin-top:10px}
pre{background:#222;color:#fff;padding:10px;overflow-x:auto}"

    h = f"
```

Gemini Archiv

```
"

for c in data:

    scr = f"
Beweis:
" if c.get(

'screenshot_path') else ""

    h += f"

    {c['title']}

    {c['url']} | {c['chat_date']}

"

    for t in c['turns']: h += f"

    {t['role']}

    {t['html']}
```

```
"  
  
h += scr + "  
  
"  
  
h += ""  
  
with open(p, "w", encoding="utf-8") as f:  
  
f.write(h)  
  
messagebox.showinfo("OK", f"Gespeichert: {p}")  
  
class SettingsDialog(tk.Toplevel):  
  
    def __init__(self, p, c, cb):  
  
        super().__init__(p);  
  
        self.cb = cb;  
  
        self.entries = {}  
  
        for i, k in enumerate(["db_host", "db_port", "db_user", "db_password", "db_name",  
                                "tor_port"]):  
  
            ttk.Label(self, text=k).grid(row=i, column=0);  
  
            e = ttk.Entry(self);  
  
            e.insert(0, c.get(k, ""));  
  
            e.grid(row=i, column=1)  
  
            if "pass" in k: e.config(show="*")  
  
            self.entries[k] = e  
  
            ttk.Button(self, text="Speichern", command=self.save).grid(row=99, columnspan=2)  
  
        def save(self):  
  
            d = {k: v.get().strip() for k, v in self.entries.items()};  
  
            save_config_to_file(d);  
  
            self.cb(d);  
  
            self.destroy()
```

```
class MainApp:

    def __init__(self, root):

        self.root = root;

        self.root.title("Universal Web Spider v20.1 (Syntax Cleaned)")

        self.root.geometry("1200x850")

        self.app_config = load_config()

        m = Menu(root);

        root.config(menu=m);

        fm = Menu(m, tearoff=0);

        m.add_cascade(label="Datei", menu=fm)

        fm.add_command(label="Einstellungen", command=self.settings)

        self.nb = ttk.Notebook(root);

        self.nb.pack(fill="both", expand=True)
```

Tab References

```
self.tabs_map = {}

self.tabs_map["crawler"] = UniversalCrawlerTab(self.nb, self)

self.nb.add(self.tabs_map["crawler"], text="🕷️ Smart Crawler & Harvester")

self.nb.add(VisuTab(self.nb, self), text="📊 Daten & Visu")

self.nb.add(GeminiTab(self.nb, self), text="🔍 Gemini Crawler")

def settings(self): SettingsDialog(self.root, self.app_config, self.update_conf)

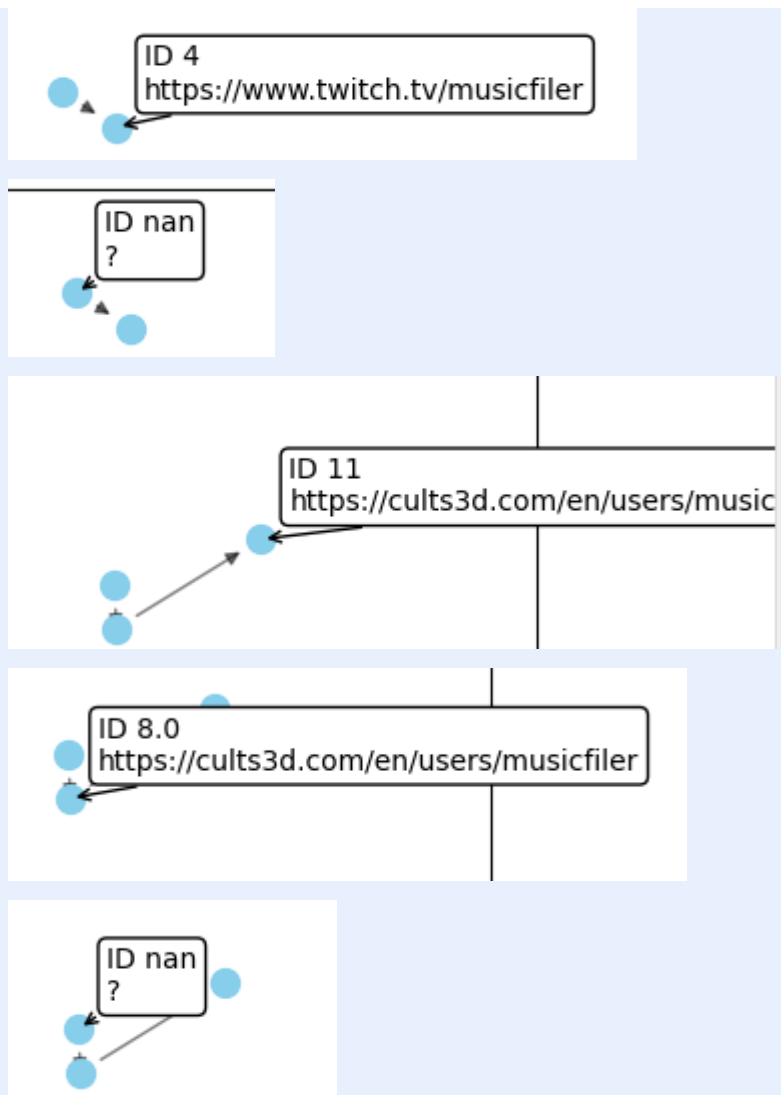
def update_conf(self, nc): self.app_config = nc; messagebox.showinfo("OK", "Gespeichert.")

if __name__ == "__main__":

    root = tk.Tk();

    MainApp(root);

    root.mainloop()
```

**MODEL**

Hier ist der optimierte Code. Ich habe die Lösungen für deine drei Hauptprobleme (Stocken bei JS, 7-Ergebnisse-Limit, ID/URL-Duplikate) direkt in die Klassen `UniversalCrawlerWorker` und `VisuTab` integriert.

Zusammenfassung der Änderungen

- 1. Infinite Scroll (DDG):** Im Such-Teil (PHASE 1) wurde eine Schleife hinzugefügt, die automatisch nach unten scrollt und auf den "Mehr laden"-Button klickt, bis das Ziel (`target_valid`) erreicht ist.
- 2. JS-Wartezeit:** `page.goto(..., wait_until="networkidle")` sorgt dafür, dass GitHub-Repos und andere React-Inhalte fertig geladen sind, bevor Text extrahiert wird.

3. **URL-Bereinigung:** URLs werden jetzt strikt am # abgeschnitten (`u.split('#')[0]`), damit Anker-Links keine Duplikate erzeugen.
4. **Datenbank/Visu Fix:** In VisuTab werden NaN Werte abgefangen und IDs explizit zu Integern konvertiert, damit der Graph sauber zeichnet.

Hier ist der vollständige, korrigierte Code (Ersetze dein Skript damit):

```
import tkinter as tk
from tkinter import ttk, scrolledtext, messagebox, Menu, filedialog,
Toplevel
import threading
import time
import sys
import webbrowser
import socket
import mysql.connector
from mysql.connector import errorcode
import html
import json
import os
import warnings
import re
import pandas as pd
import trafilatura
from langdetect import detect, detect_langs, LangDetectException
import networkx as nx
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib.pyplot as plt
import asyncio
import base64
from datetime import datetime
from bs4 import BeautifulSoup

# --- 1. CRITICAL SETUP & GLOBALS ---
sys.stdout.reconfigure(encoding='utf-8')
os.environ["PYTHONWARNINGS"] = "ignore"
warnings.simplefilter(action='ignore', category=FutureWarning)
warnings.simplefilter(action='ignore', category=UserWarning)

# Globaler Pfad für Chrome Profil
CHROME_USER_DATA_DIR = os.path.join(os.getenv('LOCALAPPDATA'), 'Google',
'Chrome',
                                'User Data') if os.name == 'nt' else
""

# --- 2. IMPORTS ---
try:
    import socks
except ImportError:
    pass

try:
```

```
from duckduckgo_search import DDGS
except ImportError:
    try:
        from ddgs import DDGS
    except ImportError:
        pass

from deep_translator import GoogleTranslator
from playwright.sync_api import sync_playwright, TimeoutError as
PlaywrightTimeout

CRAWL4AI_AVAILABLE = False
try:
    from crawl4ai import AsyncWebCrawler, BrowserConfig,
    CrawlerRunConfig, CacheMode
    CRAWL4AI_AVAILABLE = True
except ImportError:
    pass

# -----
# 3. KONFIGURATION
# -----
CONFIG_FILE = "crawler_config.json"
DEFAULT_CONFIG = {
    "db_host": "localhost", "db_port": "3306",
    "db_user": "root", "db_password": "",
    "db_name": "search_index_v3", "tor_port": "9050"
}

REGION_MAPPING = {
    'de': {'ddg': 'de-de', 'locale': 'de-DE', 'header': 'de-
DE,de;q=0.9,en-US;q=0.8,en;q=0.7'},
    'en': {'ddg': 'us-en', 'locale': 'en-US', 'header': 'en-
US,en;q=0.9'},
    'fr': {'ddg': 'fr-fr', 'locale': 'fr-FR', 'header': 'fr-
FR,fr;q=0.9,en;q=0.8'},
    'es': {'ddg': 'es-es', 'locale': 'es-ES', 'header': 'es-
ES,es;q=0.9,en;q=0.8'},
    'it': {'ddg': 'it-it', 'locale': 'it-IT', 'header': 'it-
IT,it;q=0.9,en;q=0.8'},
    'cn': {'ddg': 'cn-zh', 'locale': 'zh-CN', 'header': 'zh-
CN,zh;q=0.9,en;q=0.8'}
}
DEFAULT_REGION_CFG = {'ddg': 'wt-wt', 'locale': 'en-US', 'header': 'en-
US,en;q=0.9'}
```

```

DOMAIN_BLACKLIST = [
    "zhihu.com", "bilibili.com", "facebook.com", "twitter.com",
    "instagram.com",
    "youtube.com", "tiktok.com", "linkedin.com", "google.com",
    "duckduckgo.com",
    "bing.com", "yahoo.com", "baidu.com", "yandex.com"
]

JUNK_PATTERNS = [
    r'/login', r'/signin', r'/signup', r'/register', r'/auth',
    r'403 forbidden', r'access denied', r'captcha', r'verify you are a
human',
    r'security check', r'cloudflare', r'{"message":', r'password',
    r'forgot-password', r'reset-password'
]

# -----
# 4. UTILS & QUALITY GATE
# -----

class ToolTip:
    def __init__(self, widget, text):
        self.widget = widget;
        self.text = text;
        self.tip_window = None
        widget.bind("<Enter>", self.show_tip);
        widget.bind("<Leave>", self.hide_tip)

    def show_tip(self, event=None):
        if self.tip_window or not self.text: return
        x, y, _, _ = self.widget.bbox("insert")
        x += self.widget.winfo_rootx() + 25;
        y += self.widget.winfo_rooty() + 25
        self.tip_window = tw = Toplevel(self.widget);
        tw.wm_overrideredirect(True);
        tw.wm_geometry(f"+{x}+{y}")
        tk.Label(tw, text=self.text, justify=tk.LEFT,
background="#ffffe0", relief=tk.SOLID, borderwidth=1,
font=("tahoma", "8", "normal")).pack(ipadx=1)

    def hide_tip(self, event=None):
        if self.tip_window: self.tip_window.destroy(); self.tip_window =
None

```

```
def load_config():
    if os.path.exists(CONFIG_FILE):
        try:
            return json.load(open(CONFIG_FILE, 'r'))
        except:
            return DEFAULT_CONFIG
    return DEFAULT_CONFIG

def save_config_to_file(data): json.dump(data, open(CONFIG_FILE, 'w'),
indent=4)

def setup_proxy(use_tor, port, log_func):
    try:
        if use_tor:
            socks.set_default_proxy(socks.SOCKS5, "localhost", port);
            socket.socket = socks.socksocket
            log_func(f"[NET] Tor Proxy ({port}) aktiv.")
        else:
            socks.set_default_proxy();
            socket.socket = socket._socketobject if hasattr(socket,
"_socketobject") else socket.socket
            log_func("[NET] Direkte Verbindung.")
    except Exception as e:
        log_func(f"[NET ERR] {e}")

# Hilfsfunktion zur URL Bereinigung (NEU)
def clean_url(url):
    if not url: return ""
    # Entfernt alles nach # (Anchor Links) und trailing slashes
    return url.split('#')[0].rstrip('/')

class QualityGate:
    @staticmethod
    def is_url_allowed(url):
        if not url or not url.startswith("http"): return False
        url_lower = url.lower()
        for d in DOMAIN_BLACKLIST:
            if d in url_lower: return False
        for p in JUNK_PATTERNS:
            if p.startswith("/") and p in url_lower: return False
        return True
```

```

    @staticmethod
    def is_junk_content(title, content):
        combined = (title + " " + content).lower()[:1500]
        for p in JUNK_PATTERNS:
            if p.replace("/", "") in combined: return True
        return False

    @staticmethod
    def contains_keyword(content, keyword):
        if not keyword or not content: return True
        return keyword.lower() in content.lower()

    @staticmethod
    def check_language(text, html_raw, target_lang):
        # 1. HTML Meta Check
        if html_raw:
            try:
                soup = BeautifulSoup(html_raw, 'html.parser')
                html_tag = soup.find('html')
                if html_tag and html_tag.has_attr('lang'):
                    if html_tag['lang'].lower().startswith(target_lang):
                        return True, "Meta-Tag Match"
            except:
                pass

        # 2. Text Analyse
        if len(text) < 150: return True, "Short (Pass)"
        try:
            langs = detect_langs(text)
            if langs[0].lang.startswith(target_lang): return True, f"Text
({langs[0].prob:.2f})"
            if target_lang in ['de', 'en', 'fr', 'es'] and langs[0].lang
in ['ru', 'zh-cn', 'ja', 'ar']:
                return False, f"Wrong Lang: {langs[0].lang}"
        except:
            pass

        return True, "Fallback"

# -----
# 5. DATABASE MANAGER
# -----
class DatabaseManager:
    def __init__(self, logger_func, config):

```

```
self.log = logger_func;
self.config = config;
self.conn = None;
self.cursor = None

def connect(self):
    try:
        self.conn = mysql.connector.connect(
            user=self.config['db_user'],
            password=self.config['db_password'],
            host=self.config['db_host'],
            port=int(self.config['db_port']),
            database=self.config['db_name'], raise_on_warnings=True
        )
        self.cursor = self.conn.cursor();
        self.create_tables();
        return True
    except mysql.connector.Error as err:
        if err.errno == errorcode.ER_BAD_DB_ERROR:
            if self._create_db(): return self.connect()
        if self.log: self.log(f"[DB ERR] {err}")
        return False

def _create_db(self):
    try:
        c = mysql.connector.connect(user=self.config['db_user'],
            password=self.config['db_password'],
            host=self.config['db_host'],
            port=int(self.config['db_port']))
        c.cursor().execute(f"CREATE DATABASE IF NOT EXISTS
{self.config['db_name']} DEFAULT CHARACTER SET utf8mb4")
        c.close();
        return True
    except:
        return False

def get_conn_pandas(self):
    try:
        return mysql.connector.connect(user=self.config['db_user'],
            password=self.config['db_password'],
            host=self.config['db_host'],
            port=int(self.config['db_port']),
            database=self.config['db_name'])
    except:
```

```
        return None

    def create_tables(self):
        try:
            self.cursor.execute(
                "CREATE TABLE IF NOT EXISTS search_runs (id INT
                AUTO_INCREMENT PRIMARY KEY, query VARCHAR(255), lang VARCHAR(10), mode
                VARCHAR(50), created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)")
            self.cursor.execute("""
                CREATE TABLE IF NOT EXISTS crawled_data
                (
                    id INT AUTO_INCREMENT PRIMARY KEY,
                    run_id INT,
                    parent_id INT DEFAULT NULL,
                    url VARCHAR(768),
                    title TEXT, description TEXT,
                    raw_content MEDIUMTEXT, escaped_content MEDIUMTEXT,
                    depth_level INT DEFAULT 0,
                    search_rank INT DEFAULT 0, is_link_list BOOLEAN DEFAULT FALSE,
                    crawled_at TIMESTAMP DEFAULT
                    CURRENT_TIMESTAMP,
                    FOREIGN KEY (run_id) REFERENCES
                    search_runs(id)
                )
            """)
            self.conn.commit()
        except:
            pass

    def url_exists(self, url, run_id):
        try:
            # Checke auf saubere URL
            clean = clean_url(url)
            self.cursor.execute("SELECT id FROM crawled_data WHERE url =
            %s AND run_id = %s", (clean, run_id))
            res = self.cursor.fetchone()
            return res[0] if res else None
        except:
            return None

    def save_node(self, run_id, parent_id, url, title, desc, content,
        depth, rank, is_list=False):
        try:
            clean = clean_url(url)
            eid = self.url_exists(clean, run_id)
```

```

        if eid: return eid, True
        esc = html.escape(content) if content else ""
        self.cursor.execute("""
                                INSERT INTO crawled_data (run_id,
parent_id, url, title, description, raw_content,
escaped_content, depth_level, search_rank, is_link_list)
                                VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s,
%s, %s)""",
                                (run_id, parent_id, clean, title, desc,
content, esc, depth, rank, is_list))
        self.conn.commit();
        return self.cursor.lastrowid, False
    except Exception as e:
        if self.log: self.log(f"[DB SAVE ERR] {e}")
        return None, False

    def close(self):
        if self.conn: self.conn.close()

# -----
# 6. UNIVERSAL CRAWLER WORKER (OPTIMIZED)
# -----
class UniversalCrawlerWorker:
    def __init__(self, cfg, log):
        self.cfg = cfg; self.log = log

    def run(self, params):
        query = params['query'];
        source = params['source'];
        langs = params['langs']
        use_tor = params['tor'];
        visible = params['visible'];
        trans = params['trans']
        smart_filter = params['smart'];
        strict_keyword = params['strict']
        mode_depth = params['mode_depth'];
        recursive = params['recursive'];
        target_valid = params['target_valid']

        tor_port = int(self.cfg.get("tor_port", 9050))
        setup_proxy(use_tor, tor_port, self.log)

        db = DatabaseManager(self.log, self.cfg)

```

```

        if not db.connect(): return

        with sync_playwright() as p:
            proxy = {"server": f"socks5://localhost:{tor_port}"} if
use_tor else None
            browser = p.chromium.launch(headless=not visible,
proxy=proxy,
                                     args=["--start-maximized", "--
disable-blink-features=AutomationControlled"])

            loop_langs = langs if source == "search" else ["direct"]

            for lang in loop_langs:
                l_cfg = REGION_MAPPING.get(lang)
                if not isinstance(l_cfg, dict): l_cfg =
DEFAULT_REGION_CFG

                context = browser.new_context(
                    viewport={"width": 1280, "height": 800},
                    locale=l_cfg['locale'], timezone_id='UTC',
                    extra_http_headers={"Accept-Language":
l_cfg['header']}
                )
                context.add_init_script("Object.defineProperty(navigator,
'webdriver', {get: () => undefined})")

                # --- BLOCK MEDIA ---
                def block_media(route):
                    if route.request.resource_type in ["image", "media",
"font"]:
                        try: route.abort()
                        except: pass
                    else:
                        try: route.continue_()
                        except: pass

                context.route("**/*", block_media)

                queue_items = []

                # PHASE 1: QUEUE FILL / SEARCH
                if source == "search":
                    self.log(f"--- SUCHE {lang.upper()} (Ziel:
{target_valid}) ---")
                    term = query

```

```
        if trans and lang != 'de':
            try: term = GoogleTranslator(source='auto',
target=lang).translate(query)
            except: pass

        try:
            db.cursor.execute("INSERT INTO search_runs
(query, lang, mode) VALUES (%s, %s, %s)",
                                (term, lang,
f"mode_{mode_depth}"));
            run_id = db.cursor.lastrowid; db.conn.commit()
        except: continue

        if visible:
            self.log("[VISUAL] Starte Browser-Suche mit
Scroll...")

            try:
                s_page = context.new_page()
                ddg_url = f"https://duckduckgo.com/?q={term}
&kl={l_cfg['ddg']}&t=h_"
                s_page.goto(ddg_url)

            s_page.wait_for_selector("input[id='search_form_input']", timeout=5000)

            # --- NEUE LOGIK: SCROLLEN FÜR MEHR
            ERGEBNISSE ---

            found_links = set()
            attempts = 0

            while len(found_links) < target_valid and
attempts < 10:

                # Links sammeln
                current_elements =
s_page.locator("a[data-testid='result-title-a']").all()
                for el in current_elements:
                    try:
                        href = el.get_attribute("href")
                        if href and
QualityGate.is_url_allowed(href):

                            found_links.add(href)
                        except: pass

                    self.log(f" -> {len(found_links)}
Kandidaten gefunden...")
```

```

        if len(found_links) >= target_valid:
            break

        # Scrollen
        s_page.keyboard.press("End")
        s_page.wait_for_timeout(1500)

        # Prüfen auf "Mehr" Button
        try:
            more_btn = s_page.locator(".result--
more__btn")

            if more_btn.is_visible():
                more_btn.click()
                s_page.wait_for_timeout(1000)
            except: pass

            attempts += 1

        # In Queue übertragen
        cnt = 0
        for href in found_links:
            queue_items.append((clean_url(href), "DDG
Result", cnt + 1, 0, None))

            cnt += 1

        s_page.close()
        self.log(f" -> Queue gefüllt mit
{len(queue_items)} URLs.")

    except Exception as e:
        self.log(f"[VISUAL SEARCH ERR] {e}")
    else:
        # API Fallback (ohne Scroll, da API paginiert)
        try:
            with DDGS() as ddgs:
                res = list(ddgs.text(term,
region=l_cfg.get('ddg', 'wt-wt'), max_results=target_valid * 2,
backend="html"))

                for idx, r in enumerate(res):
                    if
QualityGate.is_url_allowed(r['href']):

queue_items.append((clean_url(r['href']), r['title'], idx + 1, 0, None))
        except Exception as e:
            self.log(f"[API ERR] {e}")

```

```

else:
    self.log(f"--- DIREKT ---")
    urls = [query] if query.startswith("http") else []
    # Falls File Input Logic existiert, hier anpassen
    if not urls: break

    try:
        db.cursor.execute("INSERT INTO search_runs
(query, lang, mode) VALUES (%s, 'raw', 'direct')",
                           (os.path.basename(query)[:
50],));

        run_id = db.cursor.lastrowid; db.conn.commit()
    except: continue

    for i, u in enumerate(urls):
queue_items.append((clean_url(u), "Start URL", i + 1, 0, None))

# PHASE 2: CRAWLING (Mit JS Wait)
visited = set()
valid_cnt = 0

while queue_items:
    if source == "search" and valid_cnt >= target_valid:
        self.log(f"*** ZIEL ERREICHT ({valid_cnt}) ***");
        break

    u, t, r, d, pid = queue_items.pop(0)

    # 1. URL BEREINIGEN
    u = clean_url(u)

    if u in visited: continue
    visited.add(u)

    if db.url_exists(u, run_id):
        self.log(f" -> [SKIP DB] {u[:30]}...")
        # Zählen wir als gefunden, um Endlosschleifen zu
vermeiden

        # valid_cnt += 1;
        continue

    self.log(f"[NODE D{d}] {u[:40]}...")
    content = ""; desc = "";
    child_links = []

```

```
        should_visit = (mode_depth >= 1) or (d > 0)

        if should_visit:
            try:
                page = context.new_page()
                page.set_extra_http_headers({"Accept-
Language": l_cfg['header']})
            try:
                # 2. LÖSUNG FÜR STOCKEN: NetworkIdle
                # Wartet bis keine Netzwerkverbindungen
                mehr offen sind (lädt JS nach)
                page.goto(u, timeout=30000,
                wait_until="networkidle")
            except PlaywrightTimeout:
                self.log(" -> Timeout (NetworkIdle),
                versuche Content trotzdem...")
            except Exception as e:
                self.log(f" -> Load Error: {e}")
                page.close(); continue

                p_content = page.content()
                p_text =
                page.evaluate("document.body.innerText")
                p_title = page.title()

                if QualityGate.is_junk_content(p_title,
                p_text):

                    self.log(" -> [DROP] Junk/Login")
                    page.close(); continue

                if source == "search" and smart_filter:
                    ok, msg =
                    QualityGate.check_language(p_text, p_content, lang)
                    if not ok:
                        self.log(f" -> [DROP] {msg}")
                        page.close(); continue

                if source == "search" and strict_keyword and
                query:

                    if not
                    QualityGate.contains_keyword(p_text, query):
                        self.log(" -> [DROP] Keyword fehlt.")
                        page.close(); continue

                if mode_depth >= 1:
```

```

        try: desc =
page.locator('meta[name="description"]').get_attribute("content") or ""
        except: pass

        # Trafilatura ist gut für Text, aber
Playwright Text ist oft "live" genauer bei JS apps
        if mode_depth >= 2:
            content = trafilatura.extract(p_content)
or p_text

        max_tree = 1 if mode_depth == 3 else 2 if
mode_depth == 4 else 0
        limit = 3 if recursive else max_tree

        if d < limit:
            hrefs =
page.evaluate("Array.from(document.links).map(a => a.href)")
            for h in hrefs:
                h_clean = clean_url(h)
                if h_clean.startswith("http") and
QualityGate.is_url_allowed(h_clean) and h_clean != u:
                    child_links.append(h_clean)
            page.close()
        except Exception as e:
            self.log(f" -> [ERR] {e}")
            try: page.close()
            except: pass

        node_id, exists = db.save_node(run_id, pid, u, t,
desc, content, d, r,

is_list=(len(child_links) > 0))

        if exists:
            self.log(" -> [DUP]");
        elif node_id:
            self.log(f" -> [SAVED] {len(content)} Chars")
            valid_cnt += 1
            if child_links:
                cnt = 0
                # Kinder hinzufügen
                for l in child_links:
                    if cnt < 15: # Limit pro Seite, um
Explosion zu vermeiden

                        queue_items.append((l, "Link", r, d +

```

```

1, node_id))

                                cnt += 1

                                context.close()
                                browser.close()
                                db.close();
                                self.log("--- FINISHED ---")

# -----
# GUI COMPONENTEN
# -----
class UniversalCrawlerTab(ttk.Frame):
    def __init__(self, parent, app):
        super().__init__(parent);
        self.app = app;
        self.setup_ui()

    def setup_ui(self):
        pnl = ttk.LabelFrame(self, text="Konfiguration", padding=10);
        pnl.pack(fill="x", padx=10, pady=5)
        self.v_src = tk.StringVar(value="search")
        r1 = ttk.Radiobutton(pnl, text="🔍 Suche", variable=self.v_src,
value="search", command=self.upd);
        r1.grid(row=0, column=0)
        Tooltip(r1, "Nutzt DuckDuckGo (API oder Browser) um URLs zu
finden.")
        r2 = ttk.Radiobutton(pnl, text="📁 Direkt/Datei",
variable=self.v_src, value="direct", command=self.upd);
        r2.grid(row=0, column=1)
        Tooltip(r2, "Crawlt eine spezifische URL oder eine Liste aus
einer .txt Datei.")
        ttk.Label(pnl, text="Input:").grid(row=0, column=2, padx=(10,
0));
        self.e_in = ttk.Entry(pnl, width=30);
        self.e_in.grid(row=0, column=3)
        self.b_file = ttk.Button(pnl, text="...", width=3,
command=self.pick, state="disabled");
        self.b_file.grid(row=0, column=4)
        ttk.Label(pnl, text="Sprachen:").grid(row=0, column=5, padx=(10,
0));
        self.e_l = ttk.Entry(pnl, width=10);
        self.e_l.insert(0, "de,en");
        self.e_l.grid(row=0, column=6)
        Tooltip(self.e_l, "Kommagetrennt (de,en,fr). Nur für Suchmodus

```

```

relevant.")

    frame_mode = ttk.Frame(pnl);
    frame_mode.grid(row=1, columnspan=7, pady=10, sticky="w")
    ttk.Label(frame_mode, text="Modus:").pack(side="left")
    self.cb_mode = ttk.Combobox(frame_mode,
                                values=["0: Nur URLs sammeln", "1: +
Meta Beschreibung", "2: + Volltext (Content)",
                                "3: + Linkliste (Level 1)",
                                "4: + Deep Links (Level 2)"], width=30);
    self.cb_mode.current(2);
    self.cb_mode.pack(side="left", padx=5)
    Tooltip(self.cb_mode,
            "Bestimmt, wie tief eine gefundene Seite analysiert wird.
\n3 & 4 erstellen Baumstrukturen.")
    self.v_rec = tk.BooleanVar(value=False);
    c_rec = ttk.Checkbutton(frame_mode, text="Rekursiv",
variable=self.v_rec);
    c_rec.pack(side="left", padx=10)
    Tooltip(c_rec, "Wenn aktiv, werden gefundene Links erneut
gescannt (entsprechend dem Modus).")
    ttk.Label(frame_mode, text="Ziel (Valid):").pack(side="left",
padx=(10, 0));
    self.s_valid = ttk.Spinbox(frame_mode, from_=1, to=1000,
width=5);
    self.s_valid.set(20);
    self.s_valid.pack(side="left")
    Tooltip(self.s_valid, "Crawler sucht so lange weiter, bis diese
Anzahl an ECHTEN Treffern erreicht ist.")

    frame_opt = ttk.Frame(pnl);
    frame_opt.grid(row=2, columnspan=7, pady=5, sticky="w")
    self.v_vis = tk.BooleanVar(value=True);
    c1 = ttk.Checkbutton(frame_opt, text="Sichtbar",
variable=self.v_vis);
    c1.pack(side="left")
    Tooltip(c1, "Zeigt den Browser an. Wichtig für Visuelle Suche.")
    self.v_tor = tk.BooleanVar(value=False);
    c2 = ttk.Checkbutton(frame_opt, text="Tor", variable=self.v_tor);
    c2.pack(side="left", padx=5)
    Tooltip(c2, "Leitet Traffic über Tor (SOCKS5).")
    self.v_tr = tk.BooleanVar(value=False);
    c3 = ttk.Checkbutton(frame_opt, text="Übersetzen",
variable=self.v_tr);
    c3.pack(side="left", padx=5)

```

```

        ToolTip(c3, "Übersetzt den Suchbegriff automatisch.")
        self.v_sm = tk.BooleanVar(value=True);
        c4 = ttk.Checkbutton(frame_opt, text="Smart Filter",
variable=self.v_sm);
        c4.pack(side="left", padx=5)
        ToolTip(c4, "Filtert falsche Sprachen und Müll-Seiten.")
        self.v_strict = tk.BooleanVar(value=True);
        c5 = ttk.Checkbutton(frame_opt, text="Zwang: Keyword",
variable=self.v_strict);
        c5.pack(side="left", padx=5)
        ToolTip(c5, "Speichert NUR, wenn das Suchwort im Text der Seite
vorkommt.")

        ttk.Button(pnl, text="🚀 STARTEN", command=self.run).grid(row=0,
column=7, rowspan=3, padx=10, sticky="nsew")
        self.log_txt = scrolledtext.ScrolledText(self, height=15,
bg="black", fg="lime");
        self.log_txt.pack(fill="both", expand=True, padx=10)

    def log(self, m):
        self.log_txt.insert(tk.END, m + "\n"); self.log_txt.see(tk.END)

    def pick(self):
        f = filedialog.askopenfilename();
        if f: self.e_in.delete(0, tk.END); self.e_in.insert(0, f)

    def upd(self):
        if self.v_src.get() == "direct":
            self.b_file.config(state="normal");
self.e_l.config(state="disabled"); self.v_strict.set(False)
        else:
            self.b_file.config(state="disabled");
self.e_l.config(state="normal")

    def run(self):
        params = {
            'query': self.e_in.get(), 'source': self.v_src.get(),
'langs': self.e_l.get().split(','),
            'tor': self.v_tor.get(), 'visible': self.v_vis.get(),
'trans': self.v_tr.get(),
            'smart': self.v_sm.get(), 'strict': self.v_strict.get(),
            'mode_depth': int(self.cb_mode.get().split(":")[0]),
            'recursive': self.v_rec.get(), 'target_valid':
int(self.s_valid.get())
        }

```

```
w = UniversalCrawlerWorker(self.app.app_config, self.log)
threading.Thread(target=w.run, args=(params,),
daemon=True).start()

class VisuTab(ttk.Frame):
    def __init__(self, parent, app):
        super().__init__(parent);
        self.app = app;
        self.df = None;
        self.setup_ui()

    def setup_ui(self):
        top = ttk.Frame(self);
        top.pack(fill="x", padx=10);
        ttk.Button(top, text="Laden",
command=self.load).pack(side="left")
        self.nb = ttk.Notebook(self);
        self.nb.pack(fill="both", expand=True)
        self.tab_tbl = ttk.Frame(self.nb);
        self.nb.add(self.tab_tbl, text="Tabelle")

        tbl_frame = ttk.Frame(self.tab_tbl);
        tbl_frame.pack(fill="both", expand=True)
        self.tree = ttk.Treeview(tbl_frame, columns=("ID", "PID", "URL",
"Title"), show="headings");
        self.tree.pack(side="left", fill="both", expand=True)
        sc = ttk.Scrollbar(tbl_frame, orient="vertical",
command=self.tree.yview);
        sc.pack(side="right", fill="y");
        self.tree.configure(yscroll=sc.set)
        for c in ("ID", "PID", "URL", "Title"): self.tree.heading(c,
text=c)
        self.tree.bind("<<TreeviewSelect>>", self.on_tbl_sel)

        self.ctx_menu = Menu(self, tearoff=0)
        self.ctx_menu.add_command(label="Deep Crawl (mit Keyword)",
command=lambda: self.launch_deep(True))
        self.ctx_menu.add_command(label="Sitemap / Full Scan (ohne
Keyword)", command=lambda: self.launch_deep(False))
        self.tree.bind("<Button-3>", self.show_ctx)

        self.txt_preview = scrolledtext.ScrolledText(self.tab_tbl,
height=10, bg="#eee");
        self.txt_preview.pack(fill="x", padx=5, pady=5)
```

```
self.tab_graph = ttk.Frame(self.nb);
self.nb.add(self.tab_graph, text="Baum-Graph")
self.canv = ttk.Frame(self.tab_graph);
self.canv.pack(fill="both", expand=True)

def load(self):
    db = DatabaseManager(None, self.app.app_config);
    conn = db.get_conn_pandas()
    if not conn: return
    self.df = pd.read_sql(
        "SELECT id, parent_id, url, title, raw_content FROM
crawled_data ORDER BY id DESC LIMIT 500", conn);
    conn.close()

    # --- FIX: NAN IN IDS ---
    # Parent ID NaN zu 0 (Root) konvertieren und alles zu Int machen
    self.df['parent_id'] = self.df['parent_id'].fillna(0).astype(int)
    self.df['id'] = self.df['id'].astype(int)

    for i in self.tree.get_children(): self.tree.delete(i)
    for _, r in self.df.iterrows(): self.tree.insert("", "end",
                                                    values=(r['id'],
r['parent_id'], r['url'], r['title']))
    self.draw_graph(self.df)

def show_ctx(self, event):
    item = self.tree.identify_row(event.y)
    if item:
        self.tree.selection_set(item);
        self.ctx_menu.post(event.x_root, event.y_root)

def launch_deep(self, use_keyword):
    sel = self.tree.selection()
    if not sel: return
    vals = self.tree.item(sel[0])['values']
    url = vals[2]
    crawler_tab = self.app.tabs_map["crawler"]
    crawler_tab.e_in.delete(0, tk.END);
    crawler_tab.e_in.insert(0, url)
    crawler_tab.v_src.set("direct");
    crawler_tab.upd()
    crawler_tab.cb_mode.current(4)
    crawler_tab.v_rec.set(True)
    crawler_tab.v_strict.set(use_keyword)
    self.app.nb.select(crawler_tab)
```

```
messagebox.showinfo("Deep Crawl", f"URL übernommen: {url}\nModus  
wurde eingestellt.\nBitte 'STARTEN' klicken.")

def on_tbl_sel(self, e):
    sel = self.tree.selection()
    if not sel: return
    iid = self.tree.item(sel[0])['values'][0]
    # Sicherstellen, dass Vergleichstypen stimmen (int vs int)
    row = self.df[self.df['id'] == int(iid)].iloc[0]
    self.txt_preview.delete("1.0", tk.END);
    self.txt_preview.insert(tk.END, row['raw_content'] or "")

def draw_graph(self, df):
    for w in self.canv.wininfo_children(): w.destroy()
    G = nx.DiGraph();
    node_map = {}
    for _, r in df.iterrows():
        G.add_node(r['id']);
        node_map[r['id']] = r['url']
        if r['parent_id'] and r['parent_id'] > 0:
            G.add_edge(r['parent_id'], r['id'])

    fig, ax = plt.subplots(figsize=(6, 5));
    # Layout etwas spreizen
    pos = nx.spring_layout(G, k=0.5, iterations=50)
    nodes = nx.draw_networkx_nodes(G, pos, ax=ax, node_size=100,
node_color='skyblue')
    nx.draw_networkx_edges(G, pos, ax=ax, alpha=0.5, arrows=True)
    annot = ax.annotate("", xy=(0, 0), xytext=(10, 10),
textcoords="offset points",
                        bbox=dict(boxstyle="round", fc="w"),
arrowprops=dict(arrowstyle="->"));
    annot.set_visible(False)

    def update_annot(ind):
        idx = ind["ind"][0];
        nid = list(G.nodes())[idx];
        url = node_map.get(nid, "?")
        annot.xy = pos[nid];
        annot.set_text(f"ID {nid}\n{url}")

    def hover(event):
        vis = annot.get_visible()
        if event.inaxes == ax:
            cont, ind = nodes.contains(event)
```

```

        if cont:
            update_annot(ind); annot.set_visible(True);
fig.canvas.draw_idle()
        else:
            if vis: annot.set_visible(False);
fig.canvas.draw_idle()

fig.canvas.mpl_connect("motion_notify_event", hover)
c = FigureCanvasTkAgg(fig, master=self.canv);
c.draw();
c.get_tk_widget().pack(fill="both", expand=True)

class GeminiTab(ttk.Frame):
    def __init__(self, parent, app):
        super().__init__(parent);
        self.app = app;
        self.setup_ui()

    def setup_ui(self):
        if not CRAWL4AI_AVAILABLE:
            ttk.Label(self, text="Crawl4AI fehlt.",
fg="red").pack(pady=20)
            return
        lbl = tk.Label(self, text="Gemini URLs:", font=("Segoe UI", 11,
"bold"));
        lbl.pack(anchor="w", padx=20, pady=(20, 5))
        self.url_input = scrolledtext.ScrolledText(self, height=10);
        self.url_input.pack(fill="x", padx=20)
        self.headless_var = tk.BooleanVar(value=False);
        tk.Checkbutton(self, text="Headless",
variable=self.headless_var).pack(anchor="w", padx=20)
        btn = tk.Button(self, text="START GEMINI",
command=self.start_thread, bg="#d32f2f", fg="white", height=2);
        btn.pack(fill="x", padx=20, pady=20)
        self.log_area = scrolledtext.ScrolledText(self, height=12,
bg="#1e1e1e", fg="#00ff00");
        self.log_area.pack(fill="x", side="bottom", padx=10, pady=10)
        if os.path.exists("urls.txt"):
            with open("urls.txt", "r", encoding="utf-8") as f:
self.url_input.insert("1.0", f.read())

    def log(self, msg):
        self.log_area.insert(tk.END, f"[{datetime.now().strftime('%H:%M:
%S')}] {msg}\n"); self.log_area.see(tk.END)

```

```

def start_thread(self):
    urls = self.url_input.get("1.0", tk.END).strip().splitlines()
    urls = [u.strip() for u in urls if "gemini.google.com" in u]
    if not urls: messagebox.showerror("Fehler", "Keine URLs.");
    return
    threading.Thread(target=self.run_async, args=(urls,),
daemon=True).start()

def run_async(self, urls):
    try:
        asyncio.run(self.crawl_process(urls))
    except Exception as e:
        self.log(f"Error: {e}")

    async def crawl_process(self, urls):
        self.log("Starte Crawl4AI...")
        js_scroll = "window.scrollTo(0, document.body.scrollHeight);
return document.body.scrollHeight;"
        out = "crawl_results";
        os.makedirs(out, exist_ok=True);
        sc_dir = os.path.join(out, "screenshots");
        os.makedirs(sc_dir, exist_ok=True)
        b_cfg = BrowserConfig(verbose=True,
headless=self.headless_var.get(), user_data_dir=CHROME_USER_DATA_DIR,
                                use_managed_browser=True)
        r_cfg = CrawlerRunConfig(cache_mode=CacheMode.BYPASS,
js_code=js_scroll, wait_for="share-viewer",
                                screenshot=True, magic=True)

        data = []
        async with AsyncWebCrawler(config=b_cfg) as crawler:
            for i, url in enumerate(urls):
                self.log(f"Lade {url}...")
                res = await crawler.arun(url=url, config=r_cfg)
                if not res.success: self.log(f"Fehler:
{res.error_message}"); continue
                spath = None
                if res.screenshot:
                    sdata = base64.b64decode(res.screenshot)
                    fname = f"chat_{int(datetime.now().timestamp())}
_{i}.png";

                    spath = os.path.join(sc_dir, fname)
                    with open(spath, "wb") as f: f.write(sdata)
                cdata = self.parse_html(res.html, url);
                cdata['screenshot_path'] = spath

```

```

        data.append(cdata);
        self.log(f"OK: {len(cdata['turns'])} Turns.")
        await asyncio.sleep(2)
        self.app.root.after(0, lambda: ReviewWindow(self.app.root, data,
self.save_report))

    def clean_html(self, el):
        if not el: return ""
        for s in ["script", "iframe", "noscript", "svg", "style"]:
            for t in el.select(s): t.decompose()
        return el.decode_contents()

    def parse_html(self, html_c, url):
        soup = BeautifulSoup(html_c, 'html.parser')
        main = soup.find('share-viewer') or soup.find('main') or
soup.body
        date = datetime.now().strftime('%d.%m.%Y');
        turns = []
        if main:
            processed = set();
            block = []
            for el in main.find_all(True):
                if el in processed: continue
                cls = el.get("class", []) or []
                if 'user-query-bubble-with-background' in cls or el.name
== 'user-query-container':
                    if block: turns.append({"role": "USER", "html":
f"<pre>{chr(10).join(block)}</pre>",
                                            "text":
chr(10).join(block)}); block = []
                    processed.add(el);
                    [processed.add(c) for c in el.find_all(True)]
                    turns.append({"role": "USER", "html":
self.clean_html(el), "text": el.get_text()})
                    elif 'query-text-line' in cls:
                        processed.add(el);
                        block.append(el.get_text())
                    elif 'message-content' in cls or (
                        el.has_attr('data-test-id') and el['data-test-
id'] == 'model-response'):
                        if block: turns.append({"role": "USER", "html":
f"<pre>{chr(10).join(block)}</pre>",
                                                "text":
chr(10).join(block)}); block = []
                        processed.add(el);

```

```

        [processed.add(c) for c in el.find_all(True)]
        turns.append({"role": "MODEL", "html":
self.clean_html(el), "text": el.get_text()})
        if block: turns.append(
            {"role": "USER", "html": f"<pre>{chr(10).join(block)}</
pre>", "text": chr(10).join(block)})
        t = "Chat Export";
        h1 = soup.find('h1')
        if h1: t = h1.get_text().strip()
        return {"title": t, "url": url, "chat_date": date, "turns":
turns}

    def save_report(self, data):
        p = filedialog.asksaveasfilename(defaultextension=".html",
filetypes=[("HTML", "*.html")])
        if not p: return
        css = "body{font-family:sans-serif;padding:
20px;background:#eee}.chat{background:white;padding:20px;margin-bottom:
20px;border-radius:8px}.role-USER{background:#e3f2fd;padding:15px;margin:
10px 0;border-left:5px solid #2196f3}.role-MODEL{background:#fff;padding:
15px;margin:10px 0;border-left:5px solid #4caf50}img.screenshot{max-
width:100%;border:1px solid #ccc;margin-top:10px}
pre{background:#222;color:#fff;padding:10px;overflow-x:auto}"
        h = f"<html><head><meta charset='utf-8'><style>{css}</style></
head><body><h1>Gemini Archiv</h1>"
        for c in data:
            scr = f"<br><b>Beweis:</b><br><img class='screenshot'
src='file:///{{c['screenshot_path'].replace(os.sep, '/')}}' />" if c.get(
'screenshot_path') else ""
            h += f"<div class='chat'><h2>{{c['title']}}</h2><p>{{c['url']}} |
{{c['chat_date']}}</p>"
            for t in c['turns']: h += f"<div class='role-
{{t['role']}}'><b>{{t['role']}}</b><br>{{t['html']}}</div>"
            h += scr + "</div>"
        h += "</body></html>"
        with open(p, "w", encoding="utf-8") as f:
            f.write(h)
        messagebox.showinfo("OK", f"Gespeichert: {p}")

class SettingsDialog(tk.Toplevel):
    def __init__(self, p, c, cb):
        super().__init__(p);
        self.cb = cb;
        self.entries = {}

```

```

        for i, k in enumerate(["db_host", "db_port", "db_user",
                                "db_password", "db_name", "tor_port"]):
            ttk.Label(self, text=k).grid(row=i, column=0);
            e = ttk.Entry(self);
            e.insert(0, c.get(k, ""));
            e.grid(row=i, column=1)
            if "pass" in k: e.config(show="*")
            self.entries[k] = e
        ttk.Button(self, text="Speichern",
                    command=self.save).grid(row=99, columnspan=2)

    def save(self):
        d = {k: v.get().strip() for k, v in self.entries.items()};
        save_config_to_file(d);
        self.cb(d);
        self.destroy()

class MainApp:
    def __init__(self, root):
        self.root = root;
        self.root.title("Universal Web Spider v20.1 (Syntax Cleaned)")
        self.root.geometry("1200x850")
        self.app_config = load_config()
        m = Menu(root);
        root.config(menu=m);
        fm = Menu(m, tearoff=0);
        m.add_cascade(label="Datei", menu=fm)
        fm.add_command(label="Einstellungen", command=self.settings)

        self.nb = ttk.Notebook(root);
        self.nb.pack(fill="both", expand=True)
        # Tab References
        self.tabs_map = {}
        self.tabs_map["crawler"] = UniversalCrawlerTab(self.nb, self)

        self.nb.add(self.tabs_map["crawler"], text="🕸 Smart Crawler &
Harvester")
        self.nb.add(VisuTab(self.nb, self), text="📊 Daten & Visu")
        self.nb.add(GeminiTab(self.nb, self), text="🔍 Gemini Crawler")

    def settings(self): SettingsDialog(self.root, self.app_config,
self.update_conf)

    def update_conf(self, nc): self.app_config = nc;

```

```
messagebox.showinfo("OK", "Gespeichert.")
```

```
if __name__ == "__main__":  
    root = tk.Tk();  
    MainApp(root);  
    root.mainloop()
```